GreenTwin — Low-Level Design

Megathon X: Singularity · Lloyds Problem Statement 2 · Team MegaMind · schema 0.4.0 · built 2026-10-11

Audience: engineers building GreenTwin, and AI coding agents working in the repo. This document is the build reference. It names the contracts, interfaces, algorithms, data and configuration each part implements. Sections 5 and 6 (contracts and interfaces) are **generated from the code** (`contracts/models.py`, `greentwin/interfaces.py`), so they always match it. The high-level view is in `SYSTEM_DESIGN.pdf`. Binding rules are in `AGENTS.md` and `contracts/`, which win if anything here disagrees.

Contents

1. The parts and how they connect
2. Conventions (all parts)
3. Repository layout
4. Part designs
5. Contracts reference (generated from contracts/models.py)
6. Interfaces reference (generated from greentwin/interfaces.py)
7. REST API (greentwin/api/app.py)
8. Data: datasets, acquisition and pipeline
9. Configuration files (contracts/)
10. Testing, evaluation, fallbacks

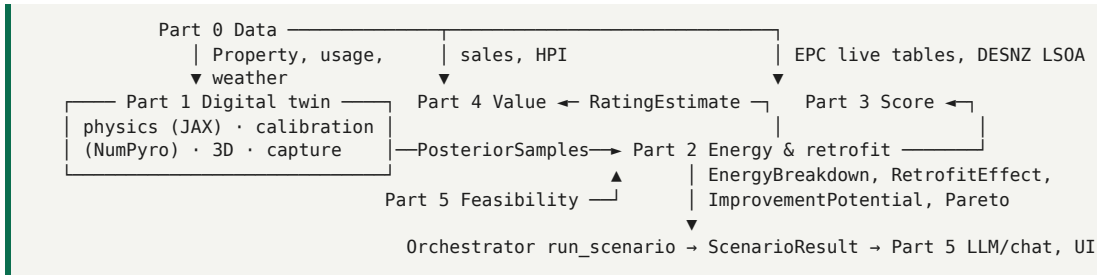
---

1. The parts and how they connect

Part	Package	Responsibility	Produces (contracts)	Consumes
0 · Data layer	`greentwin/data`, `greentwin/geo`	Download, clean, map, split and store all data; sourced factors	`Property`, `EnergyUsageMonth`, `SaleRecord`, `WeatherMonth`, `AreaEPCStats`, `LSOAConsumption`, `LoadProfileArchetype`, `EPCRecommendation`, `ReferenceLabel`, `DatasetMeta`	Public datasets, customer input
1 · Digital twin	`greentwin/twin` (JAX), `greentwin/bayes` (NumPyro), `web/twin3d`	Physics of each home, Bayesian calibration on its meters, 3D scene, photo-to-twin capture	`TwinParams`, `CalibrationResult`, `PosteriorSamples*`, `ElementHeatLoss`, `TwinScene`, `CaptureResult`, `ElectricityProfileSummary`	`Property`, usage, weather, `EnergyPrior`, load profiles
2 · Energy & retrofit engine	`greentwin/models` (EBM), `greentwin/retrofit`, `greentwin/optimise`	Energy prediction, retrofit ladder, savings, carbon, bills, improvement potential, rating after, Pareto	`EnergyPrior`, `EnergyBreakdown`, `RetrofitEffect`, `ImprovementPotential`, `RatingEstimate`, `ParetoPoint`	`PosteriorSamples`, `FeasibilityReport`, catalogue, factors, tariffs
3 · Green Home Score	`greentwin/score`	Score, band, contributions, key issues, projected score	`ScoreResult`, `KeyIssue`	`Property`, `EnergyBreakdown`, `RatingEstimate`, `AreaEPCStats`
4 · Property value	`greentwin/value`	Indicative value range before vs after; comparable greener homes	`ValueImpact`, `GreenTwin`	`Property`, `SaleRecord` (training), `RatingEstimate`
5 · Feasibility, Jev, LLM	`greentwin/retrofit/rules.py`, `greentwin/jev`, `greentwin/llm`, `greentwin/kb`	Rule vetoes + Jev feasibility; what-if routing; tool-only explanation and chat; memory	`FeasibilityReport`, `WhatIfRoute`, `ChatResponse`, `ExplanationCheck`, `CustomerMemory`	`Property`, catalogue, `ScenarioResult` (through tools)
Orchestrator	`greentwin/api`	`run_scenario`, caching, REST API, dashboard aggregation	`ScenarioResult`, `DashboardArea`	All parts through their interfaces

* `PosteriorSamples` is an in-process dataclass (arrays), not a wire contract. It is persisted to Parquet and summarised into `Distribution`s before leaving Python.

Coupling. Parts 1 and 2 form the **coupled core**: everything numeric runs through the twin. Parts 0, 3, 4 and 5 are **independent**. Each codes against its interface and a stub, and plugs into the core at four points: 1. EBM prior → calibration prior 2. feasibility list → ladder 3. estimated rating → projected score and value "after" 4. `ScenarioResult` → LLM and UI


Part 0 Data
Property, usage,
weather
sales, HPI
EPC live tables, DESNZ LSOA
Part 1 Digital twin
physics (JAX) · calibration
(NumPyro) · 3D · capture
Part 4 Value
RatingEstimate
Part 3 Score
PosteriorSamples
Part 2 Energy & retrofit
EnergyBreakdown, RetrofitEffect,
ImprovementPotential, Pareto
Part 5 Feasibility
Orchestrator run_scenario → ScenarioResult → Part 5 LLM/chat, UI

2. Conventions (all parts)

- **Units:** kWh/yr, GBP, kgCO2e, m², W/m²K, W/K. Field names carry other units explicitly (`_kwh_m2`, `_pct`, `_t`).
- **Savings = baseline − after**, per fuel. Negative = increase (electricity after a heat pump). Packages are one twin run, never a sum.
- **Uncertainty:** every uncertain output is a `Distribution`, built only with `contracts.summarise.summarise(samples, unit, tails=…, worst_tail=…)`. Use `exact()` for known values and `category_probabilities()` for bands. Fixed constants (weights, emission factors, tariffs, grants, rule vetoes) are bare numbers.
- **Missing ≠ 0:** `None` everywhere; `summarise` rejects NaN. Missing months are latent in calibration and flagged.
- **Provenance** on every result: engine, model version, data version, seed, sample count.
- **Leakage:** `epc.potential_*` are labels only. Splits are assigned once by customer, else property (`Property.split`); spatial-block splits for area models.
- **Seeds:** 42 by default; recorded in every `metrics.json`. **Schema version:** `contracts.models.SCHEMA_VERSION`, bumped on any breaking change.
- **Generated files:** `contracts/types.ts` (`python -m contracts.export_ts`) and `contracts/schema/*.json` (`python -m contracts.export_schemas`). A test fails if `types.ts` is stale.

3. Repository layout

```
contracts/              models.py (source of truth) · summarise.py · export_ts.py · export_schemas.py
                        types.ts (generated) · schema/ (generated) · examples/ (SH1–SH3 + mocks)
                        score_mapping.json · feasibility_rules.json · retrofit_catalogue.json
                        sap_estimator.json · emission_factors.json · tariffs.json · green_profits_assumptions.json
greentwin/interfaces.py Protocols for every part (section 6)
greentwin/data/         download_*.py, epc_mapper.py, splits.py, store.py (DuckDB), live_tables.py
greentwin/geo/          postcode→LSOA, boundaries, adjacency graph, hotspots
greentwin/twin/         physics.py (JAX), params.py (RdSAP defaults), electricity.py, scene.py, capture.py, generate.py
greentwin/bayes/        model.py (NumPyro), calibrate.py, hierarchy.py
greentwin/models/       energy_prior.py (EBM), gnn_lsoa.py (GraphSAGE)
greentwin/retrofit/     rules.py, ladder.py, potential.py, rating.py, catalogue.py
greentwin/optimise/     pareto.py
greentwin/score/        score.py, issues.py
greentwin/value/        hedonic.py (NumPyro), green_twins.py
greentwin/jev/          client.py, router.py, feasibility.py, calibration.py
greentwin/llm/          chat.py, tools.py, validator.py, template.py, prompts/
greentwin/kb/           sql.py, vector.py, graph.py
greentwin/api/          app.py (FastAPI), orchestrator.py, cache.py, dashboard.py
web/                    React app: twin3d/ (react-three-fiber), capture/ (react-konva), app/, dashboard/
scripts/                train_<model>.py --seed 42, make_showcase_examples.py, eval_all.py
data/                   raw/ interim/ processed/ (git-ignored) · greentwin.duckdb
artifacts/<model>/<ver>/ model.*, metrics.json, config.json      eval/ scenarios/, reports/
```

4. Part designs

4.0 Part 0 — Data layer

Pipeline: `data/raw` (as published, never edited) → `scripts/download_*.py` → `data/interim` (filtered to the North West, South West and London) → `greentwin/data/*` mappers → `data/processed/*.parquet` + `*.meta.json` (`DatasetMeta`) → `splits.py` → DuckDB `data/greentwin.duckdb`. Every other part reads only through `DataLayer` (section 6).

EPC record → `Property` mapper (`epc_mapper.py`). Free-text EPC descriptions become contract enums through ordered regex rules. Every unmatched value is logged, with a target of ≥ 95% mapped per field.

EPC field	Contract field	Rule (examples)
PROPERTY_TYPE, BUILT_FORM	property_type, built_form	House/Bungalow/Flat/Maisonette; "Mid-Terrace" → mid_terrace
CONSTRUCTION_AGE_BAND	construction_age_band	"England and Wales: 1900-1929" → 1900-1929
TOTAL_FLOOR_AREA, NUMBER_HABITABLE_ROOMS	floor_area_m2, habitable_rooms	numeric; drop < 15 m² or > 1,000 m²
WALLS_DESCRIPTION	walls.type, walls.insulation, walls.u_value	"Cavity wall, as built, no insulation (assumed)" → cavity/none; "Solid brick, with internal insulation" → solid_brick/internal; "Average thermal transmittance 0.28 W/m²K" → u_value
ROOF_DESCRIPTION	roof.type, roof.loft_insulation_mm	"Pitched, 270 mm loft insulation" → pitched_loft/270; "(another dwelling above)" → another_dwelling_above
FLOOR_DESCRIPTION	floor.type, floor.insulated	"Suspended, no insulation (assumed)"; "(another dwelling below)"
WINDOWS_DESCRIPTION, MULTI_GLAZE_PROPORTION, GLAZED_TYPE	glazing[]	"Fully double glazed" + "double glazing installed during or after 2002" → double_post_2002 100%; partial → two entries by proportion
MAINHEAT_DESCRIPTION, MAIN_FUEL	heating.main_system, heating.main_fuel	"Boiler and radiators, mains gas" + condensing flag; "Electric storage heaters"; "Air source heat pump"
MAINHEATCONT_DESCRIPTION	controls[]	"Programmer, room thermostat and TRVs" → programmer, room_thermostat, trvs
LOW_ENERGY_LIGHTING	low_energy_lighting_pct	numeric %
PHOTO_SUPPLY, SOLAR_WATER_HEATING_FLAG	renewables	PV share of roof → kWp estimate; solar thermal flag
CURRENT_ENERGY_RATING, CURRENT_ENERGY_EFFICIENCY, ENERGY_CONSUMPTION_CURRENT	epc.current_band, epc.sap_score, epc.energy_consumption_kwh_m2	numeric; the modelled energy is a validation target only
POTENTIAL_ENERGY_RATING, POTENTIAL_ENERGY_EFFICIENCY	epc.potential_*	**leakage: label only**
MAINS_GAS_FLAG	constraints.off_gas_grid	N → off-gas (primary off-gas indicator; postcode lists only as fallback)
FLOOR_HEIGHT, GLAZED_AREA, NUMBER_HEATED_ROOMS	TwinParams.volume_m3, window_area_m2	storey height × floor area; glazed-area band → window-to-floor ratio
HEATING_COST_CURRENT, HOT_WATER_COST_CURRENT, LIGHTING_COST_CURRENT	epc.estimated_annual_energy_cost_gbp	sum; validation target for standard mode
TENURE, FLOOR_LEVEL, LODGEMENT_DATE, TRANSACTION_TYPE, POSTCODE, UPRN, LOCAL_AUTHORITY	tenure, constraints.floor_level, epc.lodgement_date, location.*	postcode → district + LSOA + LA code (ONSPD), then the full postcode is dropped; UPRN is a private join key; latest certificate per UPRN = current state
CERTIFICATE_NUMBER	private key	links certificates to recommendations (EPCRecommendation); replaces the old LMK_KEY

Field names are from the EPC service's domestic data dictionary (`/download/data-dictionary?property_type=domestic`, checked 11 Oct 2026). That dictionary covers the CSV download only; API JSON names differ.

Splits (`splits.py`): `hash(customer_id or property_id, seed=42)` → 70/15/15, written once to `Property.split` and copied to every row of that home (usage, sales, labels). Spatial-block splits by postcode district are used for area models (GNN). Test: no id appears in two splits.

DuckDB tables (one per contract; nested contracts stored as JSON, plus flattened key columns):

Table	Key	Key columns	Contract
`properties`	property_id	split, lsoa_code, local_authority_code, region, type, form, age_band, floor_area_m2, sap, band, json	`Property`
`usage_monthly`	property_id, month	electricity_import_kwh, gas_kwh, pv, export, estimated_read	`EnergyUsageMonth`
`sales`	sale_id	property_id, date, price_gbp, band, sap, new_build, tenure, lsoa, la, type, floor_area, split	`SaleRecord`
`weather_monthly`	area_code, month	hdd_15_5, cdd_22, mean_temp_c, ghi, source	`WeatherMonth`
`epc_recommendations`	property_id, order	measure_code, catalogue_id	`EPCRecommendation`
`area_epc_stats`	area_code, period, dwellings	lodgements, floor area, band A–G, mean kWh/m², CO2, cost, type counts	`AreaEPCStats`
`lsoa_consumption`	lsoa_code, year, fuel	meters, mean_kwh, median_kwh	`LSOAConsumption`
`load_profiles`	archetype_id, season, day_type	shares[48], n_households	`LoadProfileArchetype`
`reference_labels`	property_id, label_type, retrofit_ids	value, unit, origin, rule_id	`ReferenceLabel`
`calibrations`	property_id, version	json	`CalibrationResult`
`posteriors` (Parquet)	property_id, version, sample	one column per `TwinParams` field	`PosteriorSamples`
`scenario_results`	scenario_id	property_id, spec_hash, model_versions, key metrics, json	`ScenarioResult`
`memory`	customer_id	json	`CustomerMemory`
`dashboard_areas`	area_code, area_type	layer columns, json	`DashboardArea`

4.1 Part 1 — Digital twin

4.1.1 Physics (JAX, `twin/physics.py`). A monthly steady-state RC model. It is written in `jax.numpy` and `jax.jit`-compiled, then `jax.vmap`ped over posterior samples. It runs inside the NumPyro model, so NUTS and SVI get exact gradients.

```
H_fabric = Σ_e U_e·A_e + y·A_exposed                    [W/K]   e ∈ walls, roof, floor, windows, doors
H_vent   = 0.33 · ach · V                               [W/K]
H        = m · (H_fabric + H_vent)                      m = heat_loss_multiplier (calibrated)
HDD*_t   = HDD15.5_t + (T_set − 19.5) · D_t · h/24       setpoint shift over D_t heating days
Q_sh,t   = max(0, H·24·HDD*_t/1000 − η_g·(G_int,t + f_sol·G_sol,t))      [kWh]
Q_hw,t   = q_hw · days_t ;  Q_base,t = b · days_t
Fuel     = Q_sh/η (boiler) or Q_sh/SCOP (heat pump) to its fuel; hot water by its source
Elec_t   = Q_base,t + pumps_t + electric heat/hot water − PV_self,t          (import)
Gas_t    = gas heat + gas hot water ;  Export_t = PV_t − PV_self,t
PV_t     = kWp · PVGIS_yield(lat, lon, azimuth, tilt)_t ;  PV_self from the electricity layer (default 0.4)
```

Modes: `calibrated` (posterior behaviour) and `standard` (SAP heating pattern and occupancy by floor area, for the rating estimator). **Retrofits:** `apply_retrofits` applies `RetrofitItem.twin_edits` (number = set, `x<k>` = multiply, string = categorical). Monotone by construction: lower U never raises heat loss, enforced by a sweep test.

Element heat loss = prior split (U·A shares) × calibrated total. Always `estimated=True` with the basis label; bills only identify the total.

4.1.2 Bayesian calibration (NumPyro, `bayes/model.py`).

Parameter	Prior	Hierarchy
`heat_loss_multiplier` m	LogNormal(μ_type,region, 0.25)	μ_type,region ~ Normal(μ_global, τ), τ ~ HalfNormal(0.2); home → LSOA → region
`setpoint_c`	Normal(19.5, 1.5), truncated [15, 24]	—
`baseload_kwh_per_day`	LogNormal(log b_0, 0.35), b_0 from the EBM electricity prior	by property type
`heating_efficiency`	Boiler: Beta around the RdSAP value; heat pump: LogNormal around the Electrification of Heat SCOP	by system
meter noise σ_f	HalfNormal(0.1) relative; ×2 for estimated reads	—

Likelihood: `y_t,f ~ StudentT(ν=5, twin_t,f(θ), σ_f·twin_t,f)` for every observed month t and fuel f. Missing months are left out of the likelihood and **predicted** from the posterior (latent, flagged).

Inference: - **Offline:** NUTS (4 chains × 1,000) on a training subset fits the hierarchy. - **Per home:** SVI (`AutoMultivariateNormal`, ~2,000 steps), initialised from the population posterior; under 1 s on CPU. Draw S = 500

samples. - **Diagnostics:** R̂ < 1.01 and ESS (NUTS); ELBO convergence (SVI); posterior-predictive coverage of observed months. - **No meters:** `method = prior_only` (prior centred on the EBM prediction, wider distributions, `data_confidence = low`).

4.1.3 3D scene (`twin/scene.py` → `TwinScene`; viewer `web/twin3d`). - **Parametric geometry:** footprint = floor area / storeys, with aspect 4:3 adjusted by built form (party walls for terraces and semis), a roof from `roof.type`, and windows from glazing share × window-to-floor ratio. - **Surfaces** carry orientation, area, U and the posterior-mean heat-loss share (colour), with the distribution available on hover. - **Retrofit slots:** PV on the best roof face, heat pump on an outside wall, loft, internal walls, plant cupboard; each slot's status comes from `FeasibilityReport`. - **Captured homes:** with a `FloorPlan`, the rooms replace the box. The viewer never computes; numbers come from `ScenarioResult`.

4.1.4 Photo-to-twin (`twin/capture.py` → `CaptureResult`; editor `web/capture`). 1. Upload a floor-plan image and room photos, with consent recorded. 2. Blur faces and personal items client-side. 3. Call **Claude Haiku 5.5** (`claude-haiku-5-5`) with image content blocks and **structured outputs** (`output_config.format` / `client.messages.parse`, JSON schema = rooms + element tags with confidence), low effort. 4. Mark fields with confidence < 0.7 as `needs_confirmation`. 5. The agent edits polygons and tags in a **react-konva** canvas. 6. Confirmed `FloorPlan` + tags fill `Property` and the twin's areas: wall area = exterior wall length × storey height − window area.

Plan area is checked against the EPC floor area (`area_vs_epc_pct`, warning above 15%). The physics stays home-level. Vision never outputs energy numbers. Always check `stop_reason` (handle `refusal`); fall back to manual tags on error.

4.1.5 Electricity layer (`twin/electricity.py` → `ElectricityProfileSummary`). - **Archetypes:** k-means (k ≈ 8) on normalised daily half-hourly profiles from the London smart-meter data, by season × day type, gives `LoadProfileArchetype`s. - **Per home:** the archetype is scaled to the calibrated baseload; heat-pump demand is distributed by degree-hours, PV by the PVGIS hourly series, and a battery is dispatched greedily (charge from surplus PV / off-peak, discharge at peak). - **Outputs:** self-consumption, export, peak share, and bills (standard, plus time-of-use once a tariff is cited). Monthly totals are reconciled to the monthly twin.

4.2 Part 2 — Energy & retrofit engine

Energy prior (EBM, `models/energy_prior.py` → `EnergyPrior`). - **Target:** annual electricity and gas kWh/m² (NEED, record-level, split by property). - **Features:** type, form, age band, floor area, region, IMD decile, main fuel, heating system, wall/roof/glazing state, degree days. - **Model:** monotone shape functions where physics requires (better insulation never raises use); conformal 90% intervals. - **Baselines:** NEED median by type × age × region, and XGBoost (`monotone_constraints`) + SHAP. - **Output:** `Distribution` per fuel plus `FeatureContribution`s (glass-box explanations).

Baseline (`baseline`). `TwinRun` over S samples → `EnergyBreakdown` via `summarise`. `intensity_kwh_m2` = **annualised metered** (grid import + gas) / floor area. Missing months come from the calibrated fit for that month's actual degree days, never ×12 and never 0. The weather-normalised intensity is a sensitivity only.

Ladder (`retrofit/ladder.py`).

```
feasible = [r for r in feasibility.items if r.status in (feasible, feasible_with_conditions)]
order    = sort(feasible, by=(ordering group from feasibility_rules.json, expected net benefit))
prev     = twin.simulate(samples, [], weather)                        # baseline, S samples
for k, r in enumerate(order, 1):
    cur  = twin.simulate(samples, order[:k], weather)                 # cumulative package
    Δ    = prev − cur   per fuel, per sample                          # marginal effect of step k
    carbon = Σ_f Δ_f · EF_f ;  bill = Σ_f Δ_f · rate_f (tariffs.json; standing charge only if a supply is removed)
    cost ~ catalogue range (null + reason if unsourced) − grants
    p_net_benefit = P(goal metric > 0 per sample) ;  p_suitable = p_feasible · p_net_benefit
    p_gain = P(Δ purchased ≥ 3% of baseline or Δscore ≥ 2)            # thresholds in config, to agree
    verdict = rules(p_suitable, p_gain, cost per kWh)  → RetrofitEffect(order=k, …)
    prev = cur
```

The marginals add up exactly to the package. A different combination is a **new run**, never "package minus a row".

Improvement potential (`retrofit/potential.py`). One run with all feasible + with-conditions retrofits. `pct = 100·(E_base − E_after)/E_base` per sample, on purchased energy. It is null + reason if E_base is 0 or missing. `pct_if_surveys_pass` adds the needs-survey items; `p_top_decile` is computed across homes in batch.

Rating estimator (`retrofit/rating.py` → `RatingEstimate`). It follows `contracts/sap_estimator.json`: 1. Run the twin in **standard** mode before and after the package, on every sample. 2. Price the energy with `tariffs.json`. 3. `ECF = 0.42·cost/(TFA + 45)`; `SAP = 117 − 121·log10(ECF)` if ECF ≥ 3.5, else `100 − 13.95·ECF`; clamp to 1–100. 4. **Anchor:** `SAP_after = SAP_recorded + (SAP_model_after − SAP_model_before)`. 5. Band from the thresholds; `band_probabilities` via `category_probabilities`.

It is labelled "estimated rating, not an EPC" and returns `None` + warning if inputs are missing or out of distribution.

Pareto (`optimise/pareto.py`). - Enumerate the feasible subsets (≤ 256, respecting `excludes` / `depends_on`). - Objectives per sample: net cost after grants (sourced costs only), CO2 cut, bill saving, score gain. - Run a non-dominated sort per sample → `p_on_front`; the knee comes from the posterior-median front. - Baselines: greedy kWh/£ and CO2-only. Metrics: hypervolume.

4.3 Part 3 — Green Home Score (`score/score.py`)

- `S = Σ_k w_k·s_k` (weights from `score_mapping.json`), rounded half-up, then band.
- **Inputs:** energy efficiency = recorded SAP (or `RatingEstimate.sap` for a projected score); intensity = `EnergyBreakdown.intensity_kwh_m2` (annualised metered); the other five come from `Property` fields.
- **Uncertainty:** evaluated per posterior sample of intensity (and SAP when estimated) → `score_distribution`, `band_probabilities`, `p_poor_or_very_poor`.
- **Key issues** (`score/issues.py`):

Issue	Rule
`high_intensity`	P(intensity > peer P50 for type × age × region) > 0.7
`poor_insulation`	insulation sub-score < 40
`old_heating`	heating sub-score < 40 or boiler age > 15
`single_glazing`	any single glazing
`no_renewables`	renewables sub-score 0
`no_controls`	controls sub-score < 40
`data_conflict` / `missing_data`	`data_quality`

Each issue carries its evidence; per-m² comparisons keep large homes from being penalised for size. - **Peer context:** local-authority band shares and mean modelled kWh/m² from the **EPC live tables** (D1, D3; `AreaEPCStats`). - **Tests:** monotonicity sweep per sub-score; every `HeatingSystem` mapped; projected ≥ current for improvement-only packages.

4.4 Part 4 — Property value (`value/hedonic.py`)

```
log(price_i / HPI_LA,type,month_i) ~ StudentT(ν, μ_i, σ_type)
μ_i = α_region + α_LA[region] + α_LSOA[LA]          pooled location effects
    + f(log floor_area) + γ_type×form + γ_age + γ_tenure + γ_rooms
    + β_region×type · SAP_i                          β ~ Normal(β₀, 0.01); β₀ from DECC 2013 / Nationwide
```

- **Data:** UCL Price Paid–EPC linked sales, split by property. Fitted with NumPyro SVI (minibatch) per region on a stratified sample; NUTS for the repeat-sales check.
- **Prediction:** V_s = current value (or posterior predictive if missing); ΔSAP_s from `RatingEstimate`; `uplift_s = exp(β_s·ΔSAP_s) − 1` → `uplift_pct`, `value_after_gbp`, `rating_effect_pct_per_sap_point`, `price_drivers` (coefficient × value).
- **Wording:** always `ValueImpact.wording`; if the 90% interval includes 0, say no clear effect.
- **Green twins:** coarsened exact matching (type, form, age band, tenure, LSOA, sale quarter; floor area ±10%) with a ≥ 15-point score gap, shown as ranges only.
- **Evaluation:** held-out coverage and pinball loss vs OLS without the rating; β vs published premiums; planted-premium recovery reported as a method check only.

4.5 Part 5 — Feasibility, Jev, LLM explanation and chat

Feasibility (`retrofit/rules.py` + `jev/feasibility.py` → `FeasibilityReport`). 1. **Layer 1:** evaluate every rule in `feasibility_rules.json` on `Property` (+ memory constraints). A `blocks` match → `infeasible`, `p_feasible = 0`. A `downgrade` → that status. **Rule vetoes always win.** 2. **Layer 2** (survivors only): Jev Choice {feasible, with conditions, needs survey, infeasible} on a state built from home facts + EPC text + agent notes + memory. Its confidence is recalibrated with isotonic regression; below τ → `needs_survey`. 3. **Without a Jev key:** Claude structured output on the same schema (`router = claude_fallback`); otherwise rules only.

Reasons shown to users come from rule codes and the knowledge base, never from the LLM.

Router (`jev/router.py` → `WhatIfRoute`). - Intent ∈ {numeric, comparative, feasibility, explain, out_of_scope}, plus `ScenarioSpec` slots, each with confidence. - τ is chosen on 30–50 labelled prompts so decisions above it are ≥ 95% correct; below τ → clarifying question. - Jev never produces kWh, £, CO2 or score values.

Chat (`llm/chat.py` → `ChatResponse`). - **Model:** `LLM_MODEL` from `.env` (team default `claude-sonnet-5-5`). - **Tools:** `get_home_facts`, `get_results`, `run_scenario`, `search_docs`, `query_graph`, defined with `strict: true` and `tool_choice: auto`. Forced tool choice is rejected on current models, so the system prompt instructs the model to call `run_scenario` for every what-if. - **Loop:** manual tool loop or the SDK tool runner. Check `stop_reason` (`refusal`, `max_tokens`) before reading content. - **Caching:** the frozen system prompt and tool list are prompt-cached. - **System prompt rules:** never compute; quote tool numbers with unit and interval; say "estimate", "projected", "indicative"; state missing, conflicting or infeasible data; no financial advice.

Validator (`llm/validator.py` → `ExplanationCheck`). - Regex-extract every number (with units, %, £, ranges) from the answer. - Each must match a number in that turn's tool outputs (relative tolerance 1% or rounding-equivalent); dates and years are allowed by an allow-list. - On failure: regenerate once with the unsupported numbers named, then fall back to

the **template explainer** (`llm/template.py`), which fills fixed sentences from `ScenarioResult` and is also used when there is no API key. - Optional layer 2: Jev "is every claim supported by this JSON?". Every call is cached by hash(prompt + tool outputs + model) in `cache/llm/`.

Knowledge base (`kb/`). - **SQL:** DuckDB views, the only source of numbers. - **Vector store:** methods docs, grant rules, EPC recommendation text; cited chunks. - **Graph:** embedded Kùzu/networkx with Home → elements, Home → LSOA → region, Retrofit → requires/blocked-by, Grant → applies-to. Queried by templated `query_graph(template_id, params)` only.

Memory: `CustomerMemory` constraints feed `FeasibilityEngine.assess`; rejected items are demoted; new bills trigger recalibration (posteriors narrow).

4.6 Orchestrator (`api/orchestrator.py`)

```
run_scenario(spec):
  prop      = data.load_property(spec.property_id)
  calib, S  = cache or twin.calibrate(prop, usage, weather, engine.energy_prior(prop))
  feas      = feasibility.assess(prop, catalogue, memory)
  base      = engine.baseline(prop, S, spec.weather)
  after     = engine.package(prop, S, spec.retrofits)              # one run
  ladder    = engine.ladder(prop, S, feas, spec.goal)
  potential = engine.improvement_potential(prop, S, feas)
  rating    = engine.estimate_rating(prop, S, spec.retrofits)
  score_b   = scorer.score(prop, base, None, peer) ; score_a = scorer.score(prop, after, rating, peer)
  value     = value_model.value_impact(prop, rating)            if spec.include_value
  pareto    = engine.pareto(prop, S, feas)                       if spec.include_pareto
  → ScenarioResult(provenance, distributions, warnings) ; cache[hash(spec, model_versions)]
```

Latency budget (showcase homes are precomputed):

Step	Budget
Calibration	cached, else < 1 s
Ladder (S = 500, ≤ 10 steps, `vmap`)	< 1 s
Pareto (≤ 256 packages)	< 3 s, or precomputed
LLM answer	typically under 10 s

5. Contracts reference (generated from `contracts/models.py`)

Schema version **0.4.0**. Wire format = JSON with these exact field names; TypeScript mirror = `contracts/types.ts` (generated). `?` = optional (may be omitted or null).

Shared primitives

`Percentiles`

Posterior percentiles; must be non-decreasing from p1 to p99.

Field	Type	Description
p1	number	
p5	number	
p10	number	
p25	number	
p50	number	
p75	number	
p90	number	
p95	number	
p99	number	

`CredibleInterval`

Central (equal-tailed) credible interval, e.g. level 0.9 = P5..P95.

Field	Type	Description
level	number	
lo	number	
hi	number	

`TailProbability`

Probability that the quantity falls beyond a meaningful threshold, e.g. P(saving < 0).

Field	Type	Description
label	string	Human-readable event, e.g. 'bill rises by more than £100/yr'
threshold	number	
direction	"below" | "above"	
probability	number	

`Histogram`

Sample histogram for the UI density plot; len(bin_edges) == len(counts) + 1.

Field	Type	Description
bin_edges	number[]	
counts	number[]	

`Distribution`

Full posterior summary of an uncertain quantity (replaces the old mean/lo/hi Interval). Built from posterior samples by `contracts.summarise.summarise`. A known-exact value (e.g. gas after a heat pump removes the boiler) is a degenerate distribution: std = 0.

Field	Type	Description
unit	string	e.g. 'kWh/yr', 'GBP/yr', 'kgCO2e/yr', 'points', '%'
n_samples	number	Posterior samples the summary was computed from
mean	number	
median	number	Equals percentiles.p50

Field	Type	Description
mode	number	Most likely value: peak of the sample density (KDE)
std	number	
skewness?	number	Sample skewness; |skew| > 0.5 -> headline uses the median
percentiles	Percentiles	
credible_intervals	CredibleInterval[]	At least the 50% and 90% intervals
tail_probabilities?	TailProbability[]	
worst_tail?	"low" | "high"	Which side is the bad outcome (low for savings, high for costs)
worst_5pct_mean	number	Mean of the worst 5% of samples (expected shortfall)
histogram?	Histogram | null	
headline?	"mean" | "median"	Which centre the UI/LLM shows first

`Provenance`

Where a number came from. Every result carries one.

Field	Type	Description
engine	"physics_l1" | "bayes_calibration" | "surrogate_deeponet" | "sap_estimator" | "ebm" | "xgboost" | "gnn" | "bayesian_hedonic" | "hedonic" | "dr_learner" | "rules" | "jev" | "claude" | "haiku_vision" | "template" | "mock"	
model_version	string	
data_version	string	
seed?	number | null	
n_samples?	number | null	Posterior samples used

`FeatureContribution`

One additive term of a glass-box model (EBM shape function or hedonic coefficient x value).

Field	Type	Description
feature	string	
value?	number | string | null	The input value for this home
contribution	number	Additive contribution in the model's output unit
unit	string	

Part 0 - Data layer (inputs, public datasets, labels)

Enum	Values
PropertyType	house, bungalow, flat, maisonette
BuiltForm	detached, semi_detached, mid_terrace, end_terrace, enclosed_mid_terrace, not_applicable
WallType	solid_brick, solid_stone, cavity, timber_frame, system_built
WallInsulation	none, cavity_filled, internal, external, as_built
RoofType	pitched_loft, pitched_room_in_roof, flat, another_dwelling_above
FloorType	suspended_timber, solid, another_dwelling_below
GlazingType	single, secondary, double_pre_2002, double_post_2002, triple
HeatingSystem	gas_boiler_condensing, gas_boiler_non_condensing, oil_boiler, lpg_boiler, electric_storage, electric_storage_hhr, electric_direct, air_source_heat_pump, ground_source_heat_pump, district_heating, communal_boiler, solid_fuel
Control	none, programmer, room_thermostat, trvs, smart_thermostat, zoned_control, smart_meter
Tenure	owner_occupied, private_rented, social_rented, unknown

`Location`

Field	Type	Description
postcode_district	string	Outward code only, e.g. 'M14'. No full postcodes/addresses.
lsoa_code?	string | null	LSOA 2021 code (ONS postcode directory)
local_authority?	string | null	Local authority name

Field	Type	Description
local_authority_code?	string | null	ONS LA code, e.g. 'E08000003'; joins EPC live tables
region	string	ONS region, e.g. 'North West'
lat?	number | null	Rounded to 3 dp (~100 m) for privacy
lon?	number | null	
altitude_m?	number | null	

EPCInfo

Field	Type	Description
current_band?	"A" | "B" | "C" | "D" | "E" | "F" | "G" | null	
sap_score?	number | null	
lodgement_date?	string | null	ISO date
potential_band?	string | null	leakage: reference label only, never a model input
potential_sap_score?	number | null	leakage: reference label only
estimated_annual_energy_cost_gbp?	number | null	EPC modelled current cost
energy_consumption_kwh_m2?	number | null	EPC modelled primary energy (kWh/m2/yr); validation target for the twin's standard mode

Walls

Field	Type	Description
type	WallType	
insulation	WallInsulation	
u_value?	number | null	W/m2K if measured; else from RdSAP defaults

Roof

Field	Type	Description
type	RoofType	
loft_insulation_mm?	number | null	Null if no loft
u_value?	number | null	

Floor

Field	Type	Description
type	FloorType	
insulated?	boolean	
u_value?	number | null	

Glazing

Field	Type	Description
type	GlazingType	
share_pct?	number	Share of windows with this glazing
window_to_floor_ratio?	number | null	

Heating

Field	Type	Description
main_system	HeatingSystem	
main_fuel	"mains_gas" | "electricity" | "oil" | "lpg" | "solid" | "district"	
system_age_years?	number | null	
seasonal_efficiency?	number | null	Boiler efficiency or heat-pump SCOP

Field	Type	Description
hot_water?	"from_main" | "electric_immersion" | "heat_pump_cylinder" | "instantaneous"	
cylinder_insulated?	boolean | null	

Renewables

Field	Type	Description
pv_kwp?	number	
pv_azimuth_deg?	number | null	180 = south
pv_tilt_deg?	number | null	
battery_kwh?	number	
solar_thermal?	boolean	

Constraints

Feasibility inputs. Rules read these; Jev reads them plus notes.

Field	Type	Description
conservation_area?	boolean	
listed_building?	boolean	
leasehold?	boolean	
has_outdoor_space?	boolean | null	For heat-pump outdoor unit
roof_access?	boolean | null	Own roof available for PV
best_roof_azimuth_deg?	number | null	
off_gas_grid?	boolean	
floor_level?	number | null	0 = ground; flats only
free_text_notes?	string[]	Agent/customer notes; read by Jev

DataQuality

Field	Type	Description
missing_fields?	string[]	
imputed_fields?	string[]	
conflicts?	string[]	e.g. 'EPC A but 400 kWh/m2 metered'
months_of_metered_data?	number	

Property

Field	Type	Description
schema_version?	string	
property_id	string	
customer_id?	string | null	Split key when present (customer-level splits)
source	"epc_real" | "synthetic" | "showcase" | "agent_capture" | "self_serve"	
split?	"train" | "val" | "test" | null	Assigned once, by property/customer
location	Location	
property_type	PropertyType	
built_form	BuiltForm	
construction_age_band	string	RdSAP band, e.g. '1900-1929'
year_built?	number | null	
storeys?	number	
floor_area_m2	number	
habitable_rooms?	number | null	
bedrooms?	number | null	
tenure?	Tenure	

Field	Type	Description
epc?	EPCInfo	
walls	Walls	
roof	Roof	
floor	Floor	
glazing	Glazing[]	
draught_proofed_pct?	number | null	
low_energy_lighting_pct?	number | null	EPC LOW_ENERGY_LIGHTING
heating	Heating	
renewables?	Renewables	
controls?	Control[]	
constraints?	Constraints	
current_value_estimate_gbp?	number | null	Baseline valuation used for value impact. NOT a historical sale price.
value_estimate_date?	string | null	
data_quality?	DataQuality	

EnergyUsageMonth

Field	Type	Description
property_id	string	
month	string	YYYY-MM
electricity_import_kwh?	number | null	Null = missing, never 0 by default
gas_kwh?	number | null	Gross CV kWh. Null = missing; 0 only if no gas supply
pv_generation_kwh?	number | null	
electricity_export_kwh?	number | null	
estimated_read?	boolean	

SaleRecord

Historical transaction (UCL Price Paid-EPC linked data). Never used as the baseline value.

Field	Type	Description
property_id	string	
date	string	
price_gbp	number	
energy_rating_at_sale?	string | null	
sap_at_sale?	number | null	
new_build?	boolean | null	
tenure?	"freehold" | "leasehold" | null	
split?	"train" | "val" | "test" | null	By property, assigned once

WeatherMonth

Field	Type	Description
area_code	string	Postcode district, LSOA or region code
month	string	
hdd_15_5	number	Heating degree days, base 15.5 C
cdd_22?	number | null	
mean_temp_c?	number | null	
ghi_kwh_m2?	number | null	Global horizontal irradiation
source	"haduk_grid" | "open_meteo_era5" | "ukcp18" | "met_office_datahub" | "synthetic"	

`DatasetMeta`

Sidecar `<file>.meta.json` written next to every processed Parquet file.

Field	Type	Description
`dataset_id`	`string`	e.g. 'epc_certificates', 'need_50k', 'epc_live_d1'
`title`	`string`	
`publisher`	`string`	
`url`	`string`	
`licence`	`string`	
`accessed`	`string`	ISO date downloaded
`release?`	`string | null`	Publisher's release/version, e.g. 'data to June 2026'
`rows`	`number`	
`split_counts?`	`Record<string, number>`	
`filters?`	`string[]`	e.g. ['regions: NW, SW, London']
`notes?`	`string[]`	Caveats: suppression, opt-outs, revisions

`EPCRecommendation`

One assessor recommendation from the EPC recommendations table. Label for recommendation checks.

Field	Type	Description
`property_id`	`string`	
`measure_code`	`string`	EPC improvement id, mapped to a catalogue id where possible
`catalogue_id?`	`string | null`	
`order?`	`number | null`	
`indicative_cost_text?`	`string | null`	
`typical_saving_text?`	`string | null`	

`AreaEPCStats`

One row of the GOV.UK 'Live tables on Energy Performance of Buildings Certificates' (MHCLG). Counts of EPCs LODGED in a period (not the dwelling stock): tables D1 (bands), D3 (floor area, modelled energy, CO2, fuel cost), EB4 (property type). Used for population calibration targets, download/mapper validation, twin standard-mode validation, peer context and dashboard layers.

Field	Type	Description
`area_code`	`string`	LA code, region name or 'E+W'
`area_type`	`"local_authority" | "region" | "england_wales" | "england" | "wales"`	
`period`	`string`	'YYYY/Q' quarter or 'YYYY' year
`dwellings?`	`"all" | "existing" | "new"`	
`lodgements`	`number`	
`total_floor_area_m2?`	`number | null`	
`band_counts?`	`Record<string, number>`	A..G + 'not_recorded' (table D1/EB1)
`mean_energy_kwh_m2?`	`number | null`	EPC modelled energy (table D3/EB3)
`mean_energy_kwh_per_dwelling?`	`number | null`	
`mean_co2_t_per_dwelling?`	`number | null`	
`mean_fuel_cost_gbp?`	`number | null`	
`property_type_counts?`	`Record<string, number>`	table EB4/NB4
`source_tables?`	`string[]`	e.g. ['D1', 'D3']
`release`	`string`	e.g. 'data to June 2026 (30 July 2026)'

`LSOAConsumption`

DESNZ sub-national domestic consumption by LSOA (real, annual). Label for the GNN area model.

Field	Type	Description
lsoa_code	string	
year	number	
fuel	"electricity" | "gas"	
meters?	number | null	
mean_kwh?	number | null	Null when suppressed
median_kwh?	number | null	

`LoadProfileArchetype`

Half-hourly electricity load shape clustered from London smart-meter data.

Field	Type	Description
archetype_id	string	
season	"winter" | "spring" | "summer" | "autumn"	
day_type	"weekday" | "weekend"	
half_hourly_share	number[]	48 values summing to 1 (share of daily kWh)
n_households	number	
source?	string	

`ReferenceLabel`

Evaluation label (PS section 9). Synthetic labels name the generator rule that made them.

Field	Type	Description
property_id	string	
label_type	"per_fuel_saving" | "package_saving" | "improvement_potential" | "feasibility" | "relevance" | "value_impact_method_check" | "rating_change"	
retrofit_ids?	string[]	
fuel?	"electricity" | "gas" | "purchased" | null	
value?	number | string | null	
unit?	string | null	
origin	"synthetic_generator" | "epc_recommendation" | "epc_history" | "rules" | "observed"	
rule_id?	string | null	Generator/rule that produced it (documented)
generator_version?	string | null	

Part 1 - Digital twin (physics, Bayesian calibration, 3D scene, photo capture)

`TwinParams`

Physics parameters of one home for the Level-1 twin (one value per parameter, i.e. one sample). Built by `params_from_property` from RdSAP defaults (+ room geometry when captured); retrofits are edits to these fields (`RetrofitItem.twin_edits`). Calibration multiplies/overrides the uncertain ones.

Field	Type	Description
property_id	string	
wall_area_m2	number	
roof_area_m2	number	
floor_area_m2	number	
window_area_m2	number	
door_area_m2?	number	
party_wall_area_m2?	number	
wall_u	number	
roof_u	number	
floor_u	number	

Field	Type	Description
window_u	number	
door_u?	number	
thermal_bridging_y?	number	W/m2K added over exposed area
air_changes_per_hour	number	
volume_m3	number	
heat_loss_multiplier?	number	Calibrated: scales total fabric + ventilation loss
setpoint_c?	number	
heating_hours_per_day?	number	
heating_efficiency	number	Boiler seasonal efficiency or heat-pump SCOP
heating_fuel	"mains_gas" | "electricity" | "oil" | "lpg" | "solid" | "district"	
hot_water_kwh_per_day	number	
baseload_kwh_per_day	number	Appliances + lighting + cooking electricity
lighting_share_of_baseload?	number	
internal_gains_w?	number	
solar_gain_factor?	number	
pv_kwp?	number	
pv_azimuth_deg?	number	
pv_tilt_deg?	number	
battery_kwh?	number	
tariff?	"standard" | "time_of_use"	

CalibrationDiagnostics

Field	Type	Description
max_r_hat?	number | null	NUTS only; < 1.01 required
min_ess?	number | null	
elbo?	number | null	SVI only
divergences?	number | null	
posterior_predictive_coverage_90?	number | null	Share of observed months in 90% PI

CalibrationResult

Bayesian calibration of one home's twin on its meter readings (summary; samples stored separately).

Field	Type	Description
property_id	string	
method	"nuts" | "svi" | "prior_only"	prior_only when no meter data exists
n_samples	number	
months_used	number	
months_imputed?	string[]	Missing months treated as latent
parameters	Record<string, Distribution>	Posterior of the calibrated parameters: heat_loss_multiplier, setpoint_c, baseload, efficiency
diagnostics?	CalibrationDiagnostics	
data_confidence	"high" | "medium" | "low"	
posterior_ref	string	Where the samples live, e.g. 'duckdb:posteriors/SH2/v3'
prior_source?	"ebm_prior" | "rdsap_defaults" | "hierarchy"	
provenance	Provenance	

`ElementHeatLoss`

Level 1 heat loss per envelope element; drives 3D colouring.

Field	Type	Description
`element`	`"roof" | "walls" | "floor" | "windows" | "doors" | "ventilation" | "party_walls"`	
`heat_loss_w_per_k`	`number`	Posterior mean
`heat_loss_distribution?`	`Distribution | null`	
`share_pct`	`number`	
`u_value?`	`number | null`	
`area_m2?`	`number | null`	
`estimated?`	`boolean`	Per-element split comes from priors, not measurement
`basis?`	`string`	Label shown wherever element colours/values are displayed

`Room`

Field	Type	Description
`room_id`	`string`	
`label`	`string`	e.g. 'living room', 'kitchen', 'bedroom 1', 'loft'
`storey?`	`number`	
`polygon_m`	`[number, number][]`	Floor polygon in metres, house-local coordinates
`area_m2`	`number`	
`ceiling_height_m?`	`number | null`	
`confidence?`	`number`	1.0 once confirmed by the agent

`FloorPlan`

Field	Type	Description
`property_id`	`string`	
`rooms`	`Room[]`	
`total_area_m2`	`number`	
`area_vs_epc_pct?`	`number | null`	(plan - EPC floor area) / EPC floor area x 100
`exterior_wall_length_m?`	`number | null`	
`window_area_m2?`	`number | null`	
`source`	`"haiku_vision" | "agent_drawn" | "parametric"`	
`confirmed_by_agent?`	`boolean`	

`ElementTag`

One element observed in a photo (vision draft or manual tag).

Field	Type	Description
`photo_id`	`string`	
`room_id?`	`string | null`	
`field`	`string`	Property field it fills, e.g. 'glazing.type', 'heating.main_system', 'roof.loft_insulation_mm'
`value`	`number | string`	
`confidence`	`number`	
`source`	`"haiku_vision" | "manual"`	
`confirmed_by_agent?`	`boolean`	

`CaptureResult`

Photo-to-twin output: drafts only. Nothing reaches the twin until the agent confirms it.

Field	Type	Description
property_id	string	
floor_plan?	FloorPlan | null	
tags?	ElementTag[]	
draft_property_fields?	Record<string, number | string>	
needs_confirmation?	string[]	Fields below the confidence threshold
vision_model?	string | null	None for manual capture
consent_recorded?	boolean	
faces_blurred?	boolean	

SceneSurface

One renderable building surface for the 3D twin.

Field	Type	Description
surface_id	string	
element	"roof" | "walls" | "floor" | "windows" | "doors" | "party_walls"	
orientation_deg?	number | null	Outward normal azimuth, 180 = south
area_m2	number	
u_value	number	
heat_loss_share_pct	number	Posterior mean share of total heat loss; drives colour
heat_loss_share?	Distribution | null	
estimated?	boolean	

RetrofitSlot

Where a retrofit object appears in the 3D scene.

Field	Type	Description
retrofit_id	string	
anchor	"roof_face" | "outside_wall" | "loft" | "wall_inner" | "windows" | "plant_cupboard" | "floor" | "room"	
surface_id?	string | null	
position_m?	[number, number, number] | null	
status	"feasible" | "feasible_with_conditions" | "needs_survey" | "infeasible"	
reason?	string | null	

TwinScene

Everything the 3D viewer needs. The scene never calculates: numbers come from ScenarioResult.

Field	Type	Description
property_id	string	
scenario_id?	string | null	Set when the scene shows a retrofit package
geometry_source	"parametric" | "floor_plan"	
property_type	PropertyType	
built_form	BuiltForm	
storeys	number	
footprint_m	[number, number]	(width, depth) of the parametric box
roof_type	RoofType	
roof_pitch_deg?	number | null	
surfaces	SceneSurface[]	
rooms?	Room[]	
retrofit_slots?	RetrofitSlot[]	
heat_loss_label?	string	

Part 2 - Energy & retrofit engine

`EnergyPrior`

EBM energy-use model output (trained on NEED): the prior for homes without meter data.

Field	Type	Description
`property_id`	`string`	
`electricity_kwh_m2`	`Distribution`	
`gas_kwh_m2`	`Distribution`	
`contributions_electricity?`	`FeatureContribution[]`	
`contributions_gas?`	`FeatureContribution[]`	
`out_of_distribution?`	`boolean`	
`provenance`	`Provenance`	

`MonthlyEnergy`

Field	Type	Description
`month`	`string`	YYYY-MM
`electricity_import_kwh`	`Distribution`	
`gas_kwh`	`Distribution`	
`imputed?`	`boolean`	True when the meter reading was missing

`EnergyBreakdown`

Field	Type	Description
`electricity_import_kwh`	`Distribution`	
`gas_kwh`	`Distribution`	
`purchased_total_kwh`	`Distribution`	
`space_heating_kwh?`	`Distribution | null`	
`hot_water_kwh?`	`Distribution | null`	
`pv_generation_kwh?`	`Distribution | null`	
`intensity_kwh_m2`	`Distribution`	Annualised METERED energy (grid import + gas) / floor area, as defined in the brief; used by the score
`intensity_weather_normalised_kwh_m2?`	`Distribution | null`	Sensitivity only; never used in the score
`monthly?`	`MonthlyEnergy[]`	Optional monthly series

`ElectricityProfileSummary`

Half-hourly electricity layer summary (smart-meter load shapes + PV + battery + tariff).

Field	Type	Description
`self_consumption_pct?`	`Distribution | null`	
`export_kwh?`	`Distribution | null`	
`peak_share_pct?`	`Distribution | null`	Share of import in 16:00-19:00
`bill_standard_gbp`	`Distribution`	
`bill_time_of_use_gbp?`	`Distribution | null`	Null until a ToU tariff is sourced
`load_archetype_id?`	`string | null`	

`RetrofitEffect`

One step of the improvement ladder: the marginal effect of adding this retrofit after the earlier steps.

Field	Type	Description
`retrofit_id`	`string`	

Field	Type	Description
order	number	Position in the recommended sequence
feasibility	Feasibility	
electricity_saving_kwh	Distribution	Marginal, in sequence; negative = increase
gas_saving_kwh	Distribution	
carbon_saving_kgco2e	Distribution	
bill_saving_gbp	Distribution	
cost_gbp?	Distribution | null	
cost_unavailable_reason?	string | null	Set when the catalogue cost is unsourced
purchased_energy_saving_kwh?	Distribution | null	Marginal grid import + gas saving
score_after?	Distribution | null	Projected score after this step (cumulative)
improvement_potential_cumulative_pct?	Distribution | null	Improvement potential reached after steps 1..order
payback_years?	Distribution | null	
p_net_benefit	number	Under ScenarioSpec.goal, cost uncertainty included
p_suitable	number	p_feasible * p_net_benefit
suitability	"High" | "Medium" | "Low"	
p_gain_above_threshold?	number | null	P(step adds >= the documented gain threshold)
verdict?	"recommended" | "optional" | "not_worth_adding" | "needs_survey" | "not_feasible" | null	
reason	string	Short human-readable reason built from rule codes/KB, not LLM

`ImprovementPotential`

PS section 3: 100 x (E_base - E_after) / E_base on purchased energy, with ALL feasible retrofits run together.

Field	Type	Description
pct?	Distribution | null	Null when the baseline is 0 or unavailable
unavailable_reason?	string | null	
electricity_change_kwh	Distribution	baseline - after; negative = increase
gas_change_kwh	Distribution	
included_retrofits?	string[]	feasible + feasible_with_conditions
excluded_needs_survey?	string[]	
pct_if_surveys_pass?	Distribution | null	
p_top_decile?	number | null	P(home in top 10% by potential), for ranking

`RatingEstimate`

Post-retrofit EPC-style rating from contracts/sap_estimator.json. Never presented as an EPC.

Field	Type	Description
sap	Distribution	
band	"A" | "B" | "C" | "D" | "E" | "F" | "G"	Mode of band_probabilities
band_probabilities?	Record<string, number>	
anchored_to_recorded_sap	boolean	False when the home has no recorded EPC
method?	"sap2012_ecf_delta"	
label?	"estimated rating, not an EPC"	

`ParetoPoint`

Field	Type	Description
package	string[]	
net_cost_gbp	Distribution	
carbon_saving_kgco2e	Distribution	
bill_saving_gbp	Distribution	
score_gain	Distribution	
p_on_front	number	
is_knee?	boolean	

Part 3 - Green Home Score

`ScoreComponent`

Field	Type	Description
name	"energy_efficiency" | "energy_intensity" | "insulation" | "heating_system" | "windows_glazing" | "renewable_energy" | "smart_energy_management"	
weight	number	
sub_score	number	
contribution	number	weight * sub_score (posterior mean if the input is uncertain)
contribution_distribution?	Distribution | null	Set when the component's input is uncertain (intensity, imputed fields)
inputs_used?	Record<string, number | string | null>	

`KeyIssue`

PS section 3: why a home performs poorly, with the evidence behind it.

Field	Type	Description
code	"high_intensity" | "poor_insulation" | "old_heating" | "single_glazing" | "no_renewables" | "no_controls" | "data_conflict" | "missing_data"	
label	string	e.g. 'High energy use per m2 for a home like this'
severity	"high" | "medium" | "low"	
evidence?	Record<string, number | string>	e.g. {'kwh_m2': 231, 'peer_p50': 165}
probability?	number | null	e.g. P(intensity above peers)

`ScoreResult`

Field	Type	Description
score	number	Rounded weighted sum (of the posterior mean inputs)
score_distribution?	Distribution | null	Score over posterior samples
band	"Highly efficient" | "Good" | "Moderate" | "Poor" | "Very poor"	Mode of band_probabilities (most probable band)
band_probabilities?	Record<string, number>	P(each band); sums to 1
p_poor_or_very_poor?	number | null	
components	ScoreComponent[]	
key_issues?	KeyIssue[]	
projected?	boolean	True for a post-retrofit score (uses RatingEstimate)
rating_source?	"recorded_epc" | "estimated" | "missing"	
peer_context?	Record<string, number | string>	e.g. local-authority band shares from EPC live table D1

Field	Type	Description
mapping_version?	string	Version of contracts/score_mapping.json used

Part 4 - Property value

`ValueImpact`

Field	Type	Description
current_value_gbp	number | null	Baseline valuation (never a historical sale price)
current_value_distribution?	Distribution | null	Set when the current value is model-estimated rather than provided
band_before?	string | null	Recorded EPC band
band_after?	string | null	From RatingEstimate (sap_estimator), not a lodged EPC
uplift_pct?	Distribution | null	
value_after_gbp?	Distribution | null	
rating_effect_pct_per_sap_point?	Distribution | null	Posterior of beta for this region x type
price_drivers?	FeatureContribution[]	Why the home is worth what it is
method	"bayesian_hedonic" | "hedonic_quantile" | "dr_learner" | "green_twin_matching" | "unavailable"	
label?	"indicative"	
wording?	string	Must accompany every display of the value range
unavailable_reason?	string | null	

`GreenTwin`

A matched greener comparable home, shown only as ranges (never an address).

Field	Type	Description
match_rank	number	
score_gap	number	Green Home Score points greener than this home
price_per_m2_gbp	Distribution	
energy_cost_gbp?	Distribution | null	
epc_band?	string | null	
same?	string[]	Matched on, e.g. ['type', 'age band', 'LSOA']

Part 5 - Feasibility, routing, LLM explanation and chat, memory

`Feasibility`

Field	Type	Description
retrofit_id	string	
status	"feasible" | "feasible_with_conditions" | "needs_survey" | "infeasible"	
decided_by	"rules" | "jev" | "claude_fallback" | "agent"	
reason_codes?	string[]	
reasons?	string[]	Human-readable, from rules/KB (never the LLM)
p_feasible	number	1.0/0.0 for rules; recalibrated confidence for Jev

`FeasibilityReport`

Field	Type	Description
property_id	string	
items	Feasibility[]	

Field	Type	Description
rules_version	string	
jev_used?	boolean	
router?	"jev" | "claude_fallback" | "rules_only"	

WhatIfRoute

Output of the System-1 router (Jev or Claude fallback).

Field	Type	Description
intent	"numeric" | "comparative" | "feasibility" | "explain" | "out_of_scope"	
confidence	number	
scenario?	ScenarioSpec | null	
needs_clarification?	boolean	
clarifying_question?	string | null	
router	"jev" | "claude_fallback"	

ExplanationCheck

Validator output for one LLM answer.

Field	Type	Description
numbers_found	string[]	
unsupported_numbers	string[]	
jev_supported_probability?	number | null	
passed	boolean	

ChatTurn

Field	Type	Description
role	"user" | "assistant"	
content	string	

Citation

Field	Type	Description
source	"tool_result" | "doc" | "rule" | "dataset"	
ref	string	scenario_id, doc id, rule id or dataset id
snippet?	string | null	

ToolCallRecord

Field	Type	Description
name	"get_home_facts" | "get_results" | "run_scenario" | "search_docs" | "query_graph"	
input	Record<string, unknown>	
output_ref?	string | null	e.g. the scenario_id a run_scenario call produced

ChatRequest

Field	Type	Description
property_id	string	
customer_id?	string | null	
message	string	
history?	ChatTurn[]	

ChatResponse

Field	Type	Description
answer	string	

Field	Type	Description
route?	WhatIfRoute | null	
tool_calls?	ToolCallRecord[]	
citations?	Citation[]	
check	ExplanationCheck	
scenario_result_id?	string | null	
fallback?	"none" | "regenerated" | "template"	
model?	string | null	LLM model id, or None for the template explainer

`CustomerMemory`

Field	Type	Description
customer_id	string	
facts?	Record<string, string>[]	e.g. {'type':'constraint','value':'renting'}
completed_retrofits?	RetrofitAction[]	
rejected?	Record<string, string>[]	{'retrofit_id':..,'reason':..}
saved_scenarios?	string[]	

Retrofit catalogue (shared config, validated by this model)

`RetrofitItem`

Field	Type	Description
id	string	
name	string	
category	"fabric" | "glazing" | "heating" | "renewables" | "controls" | "airtightness" | "lighting" | "tariff"	
twin_edits	Record<string, number | string>	TwinParams edits: a number sets the field; a string 'x' multiplies it (e.g. 'x0.25'); other strings set a categorical field (e.g. tariff)
blocked_by_rules?	string[]	Rule ids that veto this retrofit
depends_on?	string[]	Should come after these (ordering)
excludes?	string[]	Mutually exclusive retrofits
cost_gbp_low?	number | null	
cost_gbp_central?	number | null	
cost_gbp_high?	number | null	
cost_basis?	string	per_home | per_m2 | per_kwp | per_window
cost_price_year?	string | null	
cost_status?	"sourced" | "secondary_source" | "unsourced"	unsourced -> no cost figure is ever shown (cost = null + reason)
lifetime_years?	number | null	
grants?	string[]	
sizing?	Record<string, number> | null	Continuous sizing bounds, e.g. {'kwp_min':1,'kwp_max':6}
changes_kwh?	boolean	False for tariff switches: bills change, energy does not
source	string	Citation for cost/impact assumptions

Orchestration - one result per what-if, and dashboard areas

`RetrofitAction`

Field	Type	Description
retrofit_id	string	
size?	number | null	e.g. kWp, mm, kW
stage_year?	number | null	Roadmap year (0 = now)

ScenarioSpec

Field	Type	Description
schema_version?	string	
scenario_id	string	
property_id	string	
retrofits?	RetrofitAction[]	Empty = baseline only
weather?	"typical" | "cold_year" | "warm_year" | "forecast_week" | "ukcp18_2050"	
n_samples?	number	Posterior samples to propagate
include_value?	boolean	
include_pareto?	boolean	
include_element_heat_loss?	boolean	
include_electricity_profile?	boolean	
seed?	number	
goal?	"bills" | "carbon"	Defines 'net benefit' for suitability/verdicts; never changes computed numbers
requested_by?	"ui" | "llm" | "batch" | "jev_router" | "eval"	

ScenarioResult

Field	Type	Description
schema_version?	string	
scenario_id	string	
property_id	string	
provenance	Provenance	
goal?	"bills" | "carbon"	
calibration?	CalibrationResult | null	
baseline	EnergyBreakdown	
after	EnergyBreakdown	
electricity_saving_kwh	Distribution	baseline - after; negative = increase
gas_saving_kwh	Distribution	
purchased_energy_saving_kwh	Distribution	
improvement_potential	ImprovementPotential	
carbon_saving_kgco2e?	Distribution | null	Null if a fuel has no sourced factor
carbon_unavailable_reason?	string | null	
emission_factors	Record<string, number>	kgCO2e/kWh from contracts/emission_factors.json, e.g. {'electricity':0.19553,'gas':0.18296}
emission_factor_source	string	
bill_saving_gbp	Distribution	
electricity_profile_after?	ElectricityProfileSummary | null	
score_before	ScoreResult	
score_after	ScoreResult	Uses rating_after (estimated) for the energy-efficiency component
rating_after?	RatingEstimate | null	
feasibility?	FeasibilityReport | null	
retrofits?	RetrofitEffect[]	The improvement ladder, in order
value	ValueImpact	
element_heat_loss_before?	ElementHeatLoss[]	
element_heat_loss_after?	ElementHeatLoss[]	
pareto?	ParetoPoint[]	
data_confidence?	"high" | "medium" | "low"	

Field	Type	Description
warnings?	string[]	Missing/conflicting data, OOD, assumptions

DashboardArea

One area on the Lloyds heat map (LSOA or local authority). Synthetic customer layers are labelled.

Field	Type	Description
area_code	string	
area_type	"lsoa" | "local_authority" | "postcode_district"	
n_homes_modelled	number	
share_green?	Distribution | null	Share with score >= 65
share_poor?	Distribution | null	
finance_demand_gbp?	Distribution | null	
carbon_potential_t?	Distribution | null	
gnn_expected_electricity_kwh?	Distribution | null	
gnn_expected_gas_kwh?	Distribution | null	
observed?	LSOAConsumption | null	
epc_stats?	AreaEPCStats | null	Official EPC live-table figures (LA level)
off_gas_grid_share?	number | null	
synthetic_customers?	number | null	Synthetic customer base; always labelled

DashboardArea

One area on the Lloyds heat map (LSOA or local authority). Synthetic customer layers are labelled.

Field	Type	Description
area_code	string	
area_type	"lsoa" | "local_authority" | "postcode_district"	
n_homes_modelled	number	
share_green?	Distribution | null	Share with score >= 65
share_poor?	Distribution | null	
finance_demand_gbp?	Distribution | null	
carbon_potential_t?	Distribution | null	
gnn_expected_electricity_kwh?	Distribution | null	
gnn_expected_gas_kwh?	Distribution | null	
observed?	LSOAConsumption | null	
epc_stats?	AreaEPCStats | null	Official EPC live-table figures (LA level)
off_gas_grid_share?	number | null	
synthetic_customers?	number | null	Synthetic customer base; always labelled

6. Interfaces reference (generated from `greentwin/interfaces.py`)

Each part implements its Protocol; callers depend only on the Protocol (stub first, real later). `PosteriorSamples` and `TwinRun` are in-process dataclasses holding arrays over posterior samples.

```
@dataclass
class PosteriorSamples:
    """Posterior draws for one home, shared in-process between Part 1 and Parts 2-4.

    ``params`` maps a TwinParams field name to an array of shape (n_samples,). Fields that were
    not calibrated hold the RdSAP/EBM prior value repeated. Persisted to Parquet at
    ``CalibrationResult.posterior_ref``; never sent over the API (send Distributions instead).
    """

    property_id: str
    n_samples: int
    params: Mapping[str, Array]
    base: TwinParams
    seed: int = 42

@dataclass
class TwinRun:
    """Vectorised twin output over samples: arrays of shape (n_samples,) or (n_samples, 12) monthly."""

    electricity_import_kwh: Array
    gas_kwh: Array
    space_heating_kwh: Array
    hot_water_kwh: Array
    pv_generation_kwh: Array
    export_kwh: Array
    monthly_electricity_kwh: Optional[Array] = None
    monthly_gas_kwh: Optional[Array] = None
    element_heat_loss_w_per_k: Mapping[str, Array] = field(default_factory=dict)
```

`DataLayer` (Part 0)

Method	Signature	Contract / behaviour
load_property	(property_id: str) -> Property	
load_usage	(property_id: str) -> list[EnergyUsageMonth]	
load_weather	(area_code: str, months: Sequence[str]) -> list[WeatherMonth]	
load_sales	(region: Optional[str] = None, split: Optional[str] = None) -> Sequence[SaleRecord]	
load_epc_recommendations	(property_id: str) -> list[EPCRecommendation]	
load_area_epc_stats	(local_authority_code: str, period: Optional[str] = None) -> list[AreaEPCStats]	
load_lsoa_consumption	(year: Optional[int] = None) -> Sequence[LSOAConsumption]	
load_load_profiles	() -> list[LoadProfileArchetype]	
catalogue	() -> list[RetrofitItem]	
emission_factors	() -> dict[str, float]	
tariffs	() -> dict[str, Any]	

`DigitalTwin` (Part 1)

Physics (JAX), Bayesian calibration (NumPyro), 3D scene and photo capture.

Method	Signature	Contract / behaviour
params_from_property	(prop: Property) -> TwinParams	RdSAP defaults (+ captured room geometry) -> one parameter set.
apply_retrofits	(params: TwinParams, actions: Sequence[RetrofitAction]) -> TwinParams	Apply RetrofitItem.twin_edits in order. Pure function; never sums savings.
simulate	(samples: PosteriorSamples, actions: Sequence[RetrofitAction], weather: Weather, mode: Literal['calibrated', 'standard'] = 'calibrated') -> TwinRun	Run the twin on every posterior sample (jax.vmap). 'standard' = SAP occupancy for ratings.

Method	Signature	Contract / behaviour
calibrate	`(prop: Property, usage: Sequence[EnergyUsageMonth], weather: Sequence[WeatherMonth], prior: Optional[EnergyPrior] = None) -> tuple[CalibrationResult, PosteriorSamples]`	NumPyro: SVI per home (NUTS offline). Missing months are latent; no meters -> prior_only.
element_heat_loss	`(samples: PosteriorSamples, actions: Sequence[RetrofitAction] = ()) -> list[ElementHeatLoss]`	Prior split scaled by the calibrated total; always estimated=True.
electricity_profile	`(samples: PosteriorSamples, actions: Sequence[RetrofitAction], profiles: Sequence[LoadProfileArchetype], tariffs: Mapping[str, Any]) -> ElectricityProfileSummary`	Half-hourly layer: load shape + heat pump + PV - battery -> self-consumption, ToU bill.
scene	`(prop: Property, samples: PosteriorSamples, actions: Sequence[RetrofitAction] = (), feasibility: Optional[FeasibilityReport] = None) -> TwinScene`	Geometry + surface heat-loss shares + retrofit slots for the 3D viewer. Never calculates savings.
capture	`(property_id: str, floor_plan_image: Optional[bytes], room_photos: Sequence[bytes]) -> CaptureResult`	Photo-to-twin: Claude Haiku 5.5 vision drafts rooms + tags with confidence; agent confirms later.

`EnergyRetrofitEngine` (Part 2)

Energy prediction, retrofit ladder, savings/carbon/bills, improvement potential, rating after, Pareto.

Method	Signature	Contract / behaviour
energy_prior	`(prop: Property) -> EnergyPrior`	EBM trained on NEED: kWh/m2 per fuel + feature contributions (prior for homes without meters).
baseline	`(prop: Property, samples: PosteriorSamples, weather: Weather = 'typical') -> EnergyBreakdown`	Per-fuel baseline as Distributions; intensity = annualised metered energy / m2.
ladder	`(prop: Property, samples: PosteriorSamples, feasibility: FeasibilityReport, goal: Goal = 'bills') -> list[RetrofitEffect]`	Feasible retrofits in fabric-first order; marginal effect of each step on every sample; verdicts.
package	`(prop: Property, samples: PosteriorSamples, actions: Sequence[RetrofitAction]) -> EnergyBreakdown`	'After' energy for a package: ONE twin run on the whole package.
improvement_potential	`(prop: Property, samples: PosteriorSamples, feasibility: FeasibilityReport) -> ImprovementPotential`	All feasible retrofits together; null + reason if the baseline is 0 or missing.
estimate_rating	`(prop: Property, samples: PosteriorSamples, actions: Sequence[RetrofitAction]) -> Optional[RatingEstimate]`	contracts/sap_estimator.json: SAP 2012 on standard mode, anchored to the recorded SAP.
pareto	`(prop: Property, samples: PosteriorSamples, feasibility: FeasibilityReport, budget_gbp: Optional[float] = None) -> list[ParetoPoint]`	Enumerate feasible packages; P(on front) across samples; knee.

`GreenHomeScorer` (Part 3)

Method	Signature	Contract / behaviour
score	`(prop: Property, energy: EnergyBreakdown, rating: Optional[RatingEstimate] = None, peer: Optional[AreaEPCStats] = None) -> ScoreResult`	Fixed weights x monotonic sub-scores (contracts/score_mapping.json). rating given -> projected score. Distribution over posterior intensity samples; key issues with evidence; peer context from EPC live tables.

`ValueModel` (Part 4)

Method	Signature	Contract / behaviour
value_impact	`(prop: Property, rating_after: Optional[RatingEstimate]) -> ValueImpact`	Bayesian hierarchical hedonic: same home at recorded vs estimated-after rating. Always 'indicative'.
green_twins	`(prop: Property, k: int = 3) -> list[GreenTwin]`	Matched greener comparable homes, as ranges (no addresses).

`FeasibilityEngine` (Part 5)

Method	Signature	Contract / behaviour
assess	`(prop: Property, catalogue: Sequence[RetrofitItem], memory: Optional[CustomerMemory] = None) -> FeasibilityReport`	Layer 1 rule vetoes (contracts/feasibility_rules.json) always win; layer 2 Jev on survivors (Claude structured output if no key); low confidence -> needs_survey.

`Router` (Part 5)

Method	Signature	Contract / behaviour
route	`(property_id: str, message: str) -> WhatIfRoute`	Jev System-1: intent + ScenarioSpec slots with calibrated confidence; below tau -> clarify.

`Explainer` (Part 5)

Method	Signature	Contract / behaviour
chat	`(request: ChatRequest) -> ChatResponse`	Tool-only LLM (get_home_facts, get_results, run_scenario, search_docs, query_graph); numeric validator; regenerate once then template fallback. Never computes a number.

`Orchestrator` (Orchestration)

Method	Signature	Contract / behaviour
run_scenario	`(spec: ScenarioSpec) -> ScenarioResult`	calibrate (cached) -> feasibility -> baseline/package/ladder -> rating -> score before/after -> value -> pareto; one ScenarioResult with provenance; cached by hash(spec + model versions).
dashboard_areas	`(layer: str, region: Optional[str] = None) -> list[DashboardArea]`	

7. REST API (`greentwin/api/app.py`)

Method & path	Request	Response	Notes
`GET /homes/{id}`	—	`Property` + latest `ScoreResult`	`get_home_facts` tool
`POST /homes`	`Property` + `EnergyUsageMonth[]`	`{property_id}`	agent capture / self-serve
`POST /capture/{id}`	multipart images	`CaptureResult`	Haiku vision drafts; agent confirms via `PUT /homes/{id}`
`POST /scenario`	`ScenarioSpec`	`ScenarioResult`	`run_scenario`; cached
`GET /homes/{id}/scene?scenario_id=`	—	`TwinScene`	3D viewer
`GET /homes/{id}/feasibility`	—	`FeasibilityReport`	rules → Jev
`GET /homes/{id}/pareto?budget=`	—	`ParetoPoint[]`	precomputed for showcase homes
`GET /homes/{id}/green-twins`	—	`GreenTwin[]`	ranges only
`POST /route`	`{property_id, message}`	`WhatIfRoute`	Jev / Claude fallback
`POST /chat`	`ChatRequest`	`ChatResponse`	tool-only LLM + validator
`GET /dashboard/areas?layer=&region=`	—	`DashboardArea[]` (GeoJSON-wrapped)	Lloyds heat map
`GET /dashboard/segments` · `POST /dashboard/green-profits`	sliders	segment table · Monte Carlo `Distributions`	scenario, labelled assumptions
`GET /memory/{customer_id}` · `POST /memory/{customer_id}`	`CustomerMemory`	`CustomerMemory`	

Errors: 404 unknown id; 422 contract validation; 200 with `null` + reason for unavailable figures (never a fake number). Mock mode serves `contracts/examples/*_MOCK.json`, which never reaches the demo.

8. Data: datasets, acquisition and pipeline

No public dataset links property features, monthly meter readings and sale prices for the same homes. Every part therefore uses **real data where it exists**, joined at the level each dataset allows. **Physics-generated synthetic data** fills only the missing piece (monthly readings per home, and the labels), and is calibrated and validated against real aggregates.

8.1 Datasets (verified 11 Oct 2026)

Dataset	Used by	Access	Auth	Licence
EPC domestic certificates + recommendations	Part 0 homes · Part 3 score inputs · Part 2 rating check (certificate history) · Part 5 rules · recommendation labels	Bulk CSV download + developer API (OpenAPI 3, bearer token)	GOV.UK One Login (register on day 1)	OGL v3 except where stated (see the service's licensing restrictions page)
EPC live tables (statistics)	Part 0 calibration targets + download check · Part 1 twin validation · Part 3 peer context · dashboard	Direct ODS download, quarterly (data to June 2026, published 30 July 2026)	None	OGL v3
NEED anonymised data 2025	Part 2 EBM energy prior (training) · Part 0 synthetic behaviour calibration and validation	Direct download: 50,000-row and 4,000,000-row property-level samples + metadata	None	OGL v3
Sub-national domestic electricity and gas by LSOA	GNN area-model labels · dashboard observed layer · synthetic area check	Direct XLSX download (LSOA domestic electricity, LSOA domestic gas)	None	OGL v3
UCL linked Price Paid + EPC, 1995–2024 (ReShare 857911)	Part 4 Bayesian hedonic value model (training) · green twins	Open download, no registration: 857911_bundle_1.zip (3 GB) + bundle_2.zip (4 GB)	None	CC BY 4.0
HM Land Registry Price Paid Data	Part 4 recent sales after Oct 2024 (needs linking to EPC)	Direct CSV (complete file, yearly, monthly)	None	OGL v3 with attribution (address data has extra rights notes)
UK House Price Index	Part 4 market adjustment (deflate prices to today)	CSV download	None	OGL v3
Smart meter energy consumption, London households (Low Carbon London)	Part 1 electricity layer: half-hourly load-shape archetypes	Direct download (combined CSV ≈ 10 GB unzipped, ~167M rows; tariff file)	None	Open (see the London Datastore page)

Dataset	Used by	Access	Auth	Licence
Open-Meteo Historical Weather API (ERA5)	Part 0/1 monthly heating degree days and irradiance per postcode district	REST GET, JSON; cache every response	None for non-commercial use (commercial needs an API key)	Data CC BY 4.0; cite Open-Meteo and ERA5 with the Copernicus acknowledgement
Met Office HadUK-Grid	Documented degree days and 30–50-year regional trends (dashboard)	CEDA archive download	CEDA registration	OGL
PVGIS 5.3 API	Part 1 PV yield (monthly PVcalc, hourly seriescalc)	REST GET at https://re.jrc.ec.europa.eu/api/v5_3/{PVcalc|seriescalc|tmy}	None (30 calls/s per IP)	Free use with attribution to EU JRC PVGIS
ONS Postcode Directory (ONSPD)	The join key: postcode → LSOA 2021, local authority, region, lat/lon	CSV download (May 2026 release; August 2026 user guide published)	None	OGL (contains OS and Royal Mail data; attribution required)
English Indices of Deprivation 2025	Features for the EBM prior, value model and GNN; dashboard	Direct download (File 1 IMD; download-all tool at deprivation.communities.gov.uk)	None	OGL v3
LSOA 2021 boundaries	Dashboard maps; LSOA adjacency graph for the GNN	GeoJSON / GeoPackage download	None	OGL
Conservation areas + listed buildings	Part 5 feasibility rules (EWI, glazing restrictions)	Direct CSV / GeoJSON / Parquet	None	OGL v3 (attribution: Historic England, Ordnance Survey)
Off-gas-grid indicator	Part 5 rules and grants (BUS off-gas uplift); dashboard layer	EPC field per home; postcode lists as XLSX/CSV	None	EPC: OGL; postcode lists: check terms (CSE 2017 listing has no licence)
Electrification of Heat heat-pump performance (SN 9050 / 9210 / 9209)	Part 1 heat-pump SCOP priors by house type	SN 9050 cleansed data: open, summary dataset downloadable; full cleansed data by FTP order	None for SN 9050 summary; SN 9049 raw data needs UKDS registration	UK Data Service terms (check per study)
English Housing Survey energy tables	Part 0 stock distributions to fill gaps in the synthetic population	Direct XLSX tables	None	OGL v3
CubiCasa5k floor plans	Part 1 photo-to-twin evaluation (room polygons, areas)	Zenodo download	None	Check the repository LICENSE; treat as non-commercial until confirmed
Octopus Energy public products API (time-of-use tariff citation)	Part 2 time-of-use bill figures (cite one published tariff)	REST GET https://api.octopus.energy/v1/products/ ; product details list tariff unit-rate links	None for product and tariff listings (consumption endpoints need a key)	Octopus API terms; cite tariff, URL and access date in tariffs.json

8.2 Start here (day 1, in this order)

1. Register GOV.UK One Login and get EPC bulk access (the slowest step); use cached samples meanwhile.
2. Download the EPC live tables D1, D3, EB1, EB3, EB4 (ODS, minutes) and build area_epc_stats.
3. Download NEED 2025 (50k now, 4M for training) with its metadata.

4. Start the UCL 857911 bundles (≈ 7 GB) and the UK HPI CSV.
5. Download ONSPD (latest), IMD 2025 and LSOA 2021 boundaries: the join keys for everything else.
6. Download DESNZ LSOA domestic electricity and gas 2010–2024.
7. Fetch Open-Meteo weather and PVGIS yields per postcode district (cache every response).
8. Download conservation-area and listed-building files from planning.data.gov.uk.
9. Later: London smart meters (10 GB, stream with DuckDB), Electrification of Heat SN 9050 summary, CubiCasa5k, one Octopus time-of-use tariff for tariffs.json.

8.3 How to get each dataset

EPC domestic certificates + recommendations

- **Publisher / page:** MHCLG, Get energy performance data service: https://get-energy-performance-data.communities.gov.uk/
- **Format / size / coverage:** CSV; bulk = certificates lodged since 2012 (incl. expired or replaced); England and Wales, per certificate
- **Key fields:** CERTIFICATE_NUMBER (key; replaces the old LMK_KEY), POSTCODE, UPRN, UPRN_SOURCE, LOCAL_AUTHORITY, PROPERTY_TYPE, BUILT_FORM, CONSTRUCTION_AGE_BAND, TOTAL_FLOOR_AREA, FLOOR_HEIGHT, WALLS_/ROOF_/FLOOR_/WINDOWS_DESCRIPTION, MULTI_GLAZE_PROPORTION, GLAZED_TYPE, GLAZED_AREA, MAINHEAT_DESCRIPTION, MAIN_FUEL, MAINS_GAS_FLAG, MAINHEATCONT_DESCRIPTION, LOW_ENERGY_LIGHTING, PHOTO_SUPPLY, SOLAR_WATER_HEATING_FLAG, CURRENT_/POTENTIAL_ENERGY_RATING and _EFFICIENCY, ENERGY_CONSUMPTION_CURRENT, CO2_EMISSIONS_CURRENT, HEATING_/HOT_WATER_/LIGHTING_COST_CURRENT, TENURE, LODGEMENT_DATE, TRANSACTION_TYPE
- **Gotchas:** Recommendations link to certificates by CERTIFICATE_NUMBER (the API embeds them in each domestic EPC); the CSV data dictionary does not apply to API JSON names; POTENTIAL_* fields are leakage (labels only); keep the latest certificate per UPRN for the current state and the history for rating validation; addresses and UPRN are private join keys, dropped after joining.

```
# 1. Sign in with GOV.UK One Login, then: https://get-energy-performance-data.communities.gov.uk/data-access-options
# 2. Bulk CSV download (filter to NW, SW, London local authorities) into data/raw/epc/
# 3. Field definitions: .../download/data-dictionary?property_type=domestic
# 4. API: bearer token via /login/authorize; OpenAPI spec at /api-technical-documentation
```

EPC live tables (statistics)

- **Publisher / page:** MHCLG, Live tables on Energy Performance of Buildings Certificates: https://www.gov.uk/government/statistical-data-sets/live-tables-on-energy-performance-of-buildings-certificates
- **Format / size / coverage:** ODS, 0.3–16 MB per table; sheets for E&W, England, Wales, region, local authority; England and Wales; local authority × quarter from 2008/4
- **Key fields:** D1/EB1: LA code, quarter, lodgements, total floor area, counts by band A–G; D3/EB3: floor area, habitable rooms, energy consumption (kWh/m²), CO2, fuel costs; EB4/NB4: lodgements by bungalow / flat & maisonette / house
- **Gotchas:** Counts of certificates lodged, not the dwelling stock (repeat certificates, opt-outs); 'all' tables mix new and existing dwellings; a Q1 2026 LA allocation error was revised in May 2026; recompute D3 averages from totals (December 2023 correction).

```
curl -L -o data/raw/live/D1.ods https://assets.publishing.service.gov.uk/media/6a69c502f23ff62c51c1a7df/D1-_Domestic_Properties.ods
curl -L -o data/raw/live/D3.ods https://assets.publishing.service.gov.uk/media/6a69c5260d360fa159c37c91/D3-_Domestic_Properties.ods
curl -L -o data/raw/live/EB4.ods https://assets.publishing.service.gov.uk/media/6a69c62b16bc92f51e1a42d9/EB4-_Existing_Domestic_Properties.ods
# python: pd.read_excel('D1.ods', engine='odf', sheet_name='D1_by_LA', header=3)
```

NEED anonymised data 2025

- **Publisher / page:** DESNZ, National Energy Efficiency Data-Framework: https://www.gov.uk/government/statistics/national-energy-efficiency-data-framework-need-anonymised-data-2025
- **Format / size / coverage:** CSV/XLSX; 50k fits a spreadsheet, 4M for training; England and Wales, property level; gas and electricity 2005–2023
- **Key fields:** annual gas and electricity consumption by year, property type, age band, floor-area band, region, IMD band, installed measures (loft, cavity, boiler) with years
- **Gotchas:** No postcode: join to EPC homes by archetype (type × age × floor area × region); values banded/rounded; a 2026 release may exist on the NEED collection page.

```
# Download the 4M-row sample + metadata from the gov.uk page into data/raw/need/
```

Sub-national domestic electricity and gas by LSOA

- **Publisher / page:** DESNZ (accredited official statistics): https://www.gov.uk/government/collections/sub-national-electricity-consumption-data
- **Format / size / coverage:** XLSX, 2010–2024 (2024 summary updated 19 Dec 2025); England and Wales LSOAs (Scotland data zones), domestic meters only
- **Key fields:** LSOA code, number of meters, total kWh, mean and median kWh per meter
- **Gotchas:** Domestic meters only; small areas suppressed (keep None, never 0); earlier years revised each release.

```
# Collection pages: sub-national electricity and gas consumption data -> LSOA domestic files into data/raw/desnz/
```

UCL linked Price Paid + EPC, 1995–2024 (ReShare 857911)

- **Publisher / page:** UCL (Chi, Dennett, Oléron-Evans, Morphet), UK Data Service ReShare: https://reshare.ukdataservice.ac.uk/857911/
- **Format / size / coverage:** Zipped CSV; >22M transactions, 106 variables (tranall_link_26122024); England and Wales, sales 1 Jan 1995 – 31 Oct 2024
- **Key fields:** price, date, priceper (price per m²), classt, lad23cd, LMK_KEY (EPC id), UPRN (post-Nov 2021), floor area, habitable rooms, EPC rating fields
- **Gotchas:** The linked file is the initial, uncleaned version: dedupe, drop non-market and outliers. ReShare 854942 (2011–19, 5.7M sales, cleaner) is a cross-check, but its linked CSV is CC BY-NC-SA (non-commercial).

```
wget -P data/raw/ucl https://reshare.ukdataservice.ac.uk/857911/   # follow the bundle links on the page
```

HM Land Registry Price Paid Data

- **Publisher / page:** HM Land Registry: https://www.gov.uk/government/statistical-data-sets/price-paid-data-downloads
- **Format / size / coverage:** CSV; complete file ≈ 5 GB; England and Wales, 1995 → latest month
- **Key fields:** transaction id, price, date, postcode, property type, new build, tenure, PPD category (A market / B other)
- **Gotchas:** No floor area or EPC (link by address/UPRN); drop category B. Standard URL, not re-verified this session.

```
# Download the yearly files for 2024–2026 from the page into data/raw/ppd/
```

UK House Price Index

- **Publisher / page:** HM Land Registry / ONS: https://landregistry.data.gov.uk/app/ukhpi
- **Format / size / coverage:** CSV; Local authority × property type × month, 1995 →
- **Key fields:** index and average price by property type
- **Gotchas:** Use the LA × type index matching each sale. Standard URL, not re-verified this session.

```
# Download full UK HPI CSV from the app's download page into data/raw/hpi/
```

Smart meter energy consumption, London households (Low Carbon London)

- **Publisher / page:** UK Power Networks via London Datastore: https://data.london.gov.uk/dataset/smartmeter-energy-consumption-data-in-london-households-vqm0d
- **Format / size / coverage:** CSV, half-hourly; 5,567 London households, Nov 2011 – Feb 2014; ~1,100 on a dynamic time-of-use tariff in 2013
- **Key fields:** household id, tariff group (Std / ToU), timestamp, kWh per half hour; ToU price schedule
- **Gotchas:** Electricity only, London only, 2011–14 (state as a limitation); stream with DuckDB; a smaller TU Delft refactored copy (CC BY) exists; UKDS SN 7857 is cleaner but restricted.

```
# Download the combined CSV from the data.gov.uk / London Datastore links; query in place with DuckDB
```

Open-Meteo Historical Weather API (ERA5)

- **Publisher / page:** Open-Meteo (Copernicus ERA5 / ERA5-Land): https://open-meteo.com/en/docs/historical-weather-api
- **Format / size / coverage:** JSON; ERA5 0.25° from 1940, ERA5-Land 0.1° from 1950
- **Key fields:** daily temperature_2m_mean, shortwave_radiation_sum (MJ/m²)
- **Gotchas:** A timezone parameter is required for daily variables; compute HDD (base 15.5 °C) from daily means.

```
curl "https://archive-api.open-meteo.com/v1/archive?latitude=53.45&longitude=-2.22&start_date=2024-01-01&end_date=2024-12-31&daily=temperature_2m_mean,shortwave_radiation_sum&timezone=Europe/London"
```

Met Office HadUK-Grid

- **Publisher / page:** Met Office via CEDA: https://catalogue.ceda.ac.uk/uuid/4dc8450d889a491ebb20e724debe2dfb
- **Format / size / coverage:** NetCDF, 1 km / 5 km / regional; UK, daily and monthly temperature
- **Key fields:** tas, tasmax, tasmin

- **Gotchas:** Large NetCDF files; use regional/5 km products for trends. Not re-verified this session.

```
# Register at CEDA, then download monthly tas for the needed years
```

PVGIS 5.3 API

- **Publisher / page:** EU Joint Research Centre: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/getting-started-pvgis/api-non-interactive-service_en
- **Format / size / coverage:** JSON / CSV; Europe incl. UK
- **Key fields:** monthly and hourly PV output per kWp for azimuth and tilt
- **Gotchas:** Use the versioned v5_3 path; PVGIS aspect 0 = south; cache per postcode district.

```
curl "https://re.jrc.ec.europa.eu/api/v5_3/PVcalc?
lat=51.48&lon=-2.52&peakpower=1&loss=14&aspect=0&angle=35&outputformat=json"
```

ONS Postcode Directory (ONSPD)

- **Publisher / page:** ONS Open Geography Portal: https://geoportal.statistics.gov.uk/
- **Format / size / coverage:** Zipped CSV ≈ 235 MB, quarterly; UK postcodes, live and terminated
- **Key fields:** pcds, lsoa21cd, local authority code, region code, lat, long (names vary slightly by release)
- **Gotchas:** Use the latest release; keep only district + LSOA after joining (privacy).

```
# Open Geography Portal -> 'ONS Postcode Directory (May 2026)' -> download CSV into data/raw/onspd/
```

English Indices of Deprivation 2025

- **Publisher / page:** MHCLG: https://www.gov.uk/government/statistics/english-indices-of-deprivation-2025
- **Format / size / coverage:** XLSX/CSV; 33,755 LSOAs (2021 geography), England; published 30 Oct 2025
- **Key fields:** LSOA code, IMD score, rank, decile, domain scores
- **Gotchas:** England only; Wales uses WIMD (separate); use the corrected LSOA → LAD lookup (Aug 2025).

```
# deprivation.communities.gov.uk/download-all -> IMD 2025 LSOA file
```

LSOA 2021 boundaries

- **Publisher / page:** ONS Open Geography Portal: https://geoportal.statistics.gov.uk/
- **Format / size / coverage:** Generalised (BGC) for maps; full resolution for adjacency; England and Wales
- **Key fields:** LSOA21CD, geometry
- **Gotchas:** Build adjacency (queen contiguity) once with geopandas/PySAL. Not re-verified this session.

```
# Open Geography Portal -> 'LSOA (December 2021) Boundaries EW BGC' -> GeoJSON
```

Conservation areas + listed buildings

- **Publisher / page:** MHCLG planning.data.gov.uk (Historic England for listed buildings): https://www.planning.data.gov.uk/dataset/conservation-area
- **Format / size / coverage:** 10,666 conservation areas from 156 providers; 382,284 listed-building points; England (incomplete: not every authority has published)
- **Key fields:** entity, name, geometry, organisation, start/end dates (listed buildings: list entry, grade, point)
- **Gotchas:** Incomplete coverage: unknown ≠ not constrained; flag the agent to confirm. Listed buildings are points inside the building, not its extent.

```
curl -L -o data/raw/planning/conservation-area.parquet https://files.planning.data.gov.uk/dataset/conservation-
area.parquet
curl -L -o data/raw/planning/listed-building.parquet https://files.planning.data.gov.uk/dataset/listed-building.parquet
```

Off-gas-grid indicator

- **Publisher / page:** EPC MAINS_GAS_FLAG (primary); Xoserve / Centre for Sustainable Energy postcode lists (fallback): https://data.london.gov.uk/dataset/postcodes-not-on-the-gas-network
- **Format / size / coverage:** Per home / postcode list; Postcode lists are dated (2013–2018)
- **Key fields:** MAINS_GAS_FLAG (Y/N); postcode, OA/LSOA (CSE)
- **Gotchas:** Prefer the EPC flag; use postcode lists only where the EPC flag is missing, and say they are dated.

```
# Use MAINS_GAS_FLAG from the EPC bulk file
```

Electrification of Heat heat-pump performance (SN 9050 / 9210 / 9209)

- **Publisher / page:** Energy Systems Catapult via UK Data Service: https://doc.ukdataservice.ac.uk/doc/9050/mrdoc/pdf/9050_eoh_Interim_insights_from_heat_pump_performance_data.pdf

- **Format / size / coverage:** CSV (742 files in the full cleansed set); daily (SN 9210) and 30-minute (SN 9209) datasets; ~750 heat pumps, Nov 2020 – Sep 2023 (2nd edition)
- **Key fields:** seasonal performance factor, daily/half-hourly heat output and electricity input, house type
- **Gotchas:** Use the summary SPF by system type and house type as the prior; analysis code on GitHub (ES-Catapult/electrification_of_heat).

```
# UK Data Service catalogue: SN 9050 -> download summary dataset
```

English Housing Survey energy tables

- **Publisher / page:** MHCLG: https://www.gov.uk/government/statistics/english-housing-survey-2022-to-2023-energy
- **Format / size / coverage:** XLSX; England dwelling stock
- **Key fields:** wall type, insulation, glazing, heating by age and type
- **Gotchas:** Survey estimates (with sampling error); used for marginals only. Not re-verified this session.

```
# Download the energy report annex tables
```

CubiCasa5k floor plans

- **Publisher / page:** CubiCasa / Aalto University (Zenodo record 2613548): https://github.com/CubiCasa/CubiCasa5k
- **Format / size / coverage:** 5,000 floor plans (4,200 / 400 / 400), SVG annotations per image; Finnish floor plans (style differs from UK plans)
- **Key fields:** room polygons and types, walls, doors, windows
- **Gotchas:** Only for evaluating extraction quality; add a small set of our own labelled UK plans.

```
# Zenodo record 2613548 -> download archive
```

Octopus Energy public products API (time-of-use tariff citation)

- **Publisher / page:** Octopus Energy: https://docs.octopus.energy/rest/guides/endpoints/
- **Format / size / coverage:** JSON, paginated; Current and historical tariffs (e.g. Agile, Go)
- **Key fields:** tariff codes, standard unit rates, standing charges, valid_from/valid_to
- **Gotchas:** Prices vary by region and day; record the exact tariff and date used. Until cited, ToU bills stay null.

```
curl https://api.octopus.energy/v1/products/
```

8.4 EPC live tables (MHCLG): processing and use

Source. GOV.UK *Live tables on Energy Performance of Buildings Certificates*: quarterly ODS files (data to June 2026, published 30 July 2026), England and Wales, built from the EPC register since 2008. Each table has sheets for England & Wales, England only, Wales only, by region and **by local authority**. The LA sheet has one row per LA code × quarter (`YYYY/Q`, from 2008/4).

Table	Columns used	Becomes
D1 (all dwellings) / EB1 (existing)	LA code, quarter, number of lodgements, total floor area, counts A–G + not recorded	`AreaEPCStats.band_counts`, `lodgements`, `total_floor_area_m2`
D3 / EB3	lodgements, total floor area, habitable rooms, energy consumption current (kWh/m² pa, total), CO2 (t pa), fuel costs	`mean_energy_kwh_m2`, `mean_energy_kwh_per_dwelling`, `mean_co2_t_per_dwelling`, `mean_fuel_cost_gbp`, **recomputed from totals ÷ counts** (the December 2023 correction note)
EB4 / NB4	lodgements and floor area for bungalow, flat & maisonette, house	`property_type_counts`

Loader (`greentwin/data/live_tables.py`): 1. `pandas.read_excel(path, sheet_name="D1_by_LA", engine="odf", header=3)`. 2. Rename columns and coerce numerics; suppressed or blank cells become `None`. 3. Melt bands; join D1 + D3 + EB4 on (LA code, quarter). 4. Write `area_epc_stats.parquet` + `meta.json` (`release`, revision notes).

Repeat for the region and England & Wales sheets with `area_type` set.

Use	Part	How
Population calibration targets	0	Rake (iterative proportional fitting) the sampled EPC records and the synthetic population so band mix (D1), property-type mix (EB4) and mean floor area per LA match the official totals for the same period
Download / mapper validation	0	Counts of our downloaded certificates per LA × quarter × band are reconciled against D1. A gap > 2% flags missing files, opt-outs or mapping errors
Twin validation	1	Twin **standard mode** (SAP-like occupancy) aggregated per LA vs D3 mean modelled kWh/m² and fuel cost: a physics check against official EPC-modelled figures

Use	Part	How
Score peer context	3	`ScoreResult.peer_context` gives the local-authority band shares and median modelled kWh/m² ("your home vs your area")
Synthetic-data validation	0	Synthetic band, type and kWh/m² distributions vs D1 / EB4 / D3 per LA (chi-square, KS), reported in the evaluation
Dashboard layers	Orchestrator	`DashboardArea.epc_stats`: share A–C vs D–G, trend over quarters, mean modelled kWh/m², CO2 per dwelling (official LA layer beside our modelled LSOA layer)

Caveats (documented in `DatasetMeta.notes`): - Counts are **lodgements in a period**, not the dwelling stock; repeat certificates and opt-outs exist. - "All" tables mix new and existing dwellings; use the EB tables for existing stock. - A Q1 2026 LA allocation error was revised on 15 and 19 May 2026. - D3 averages must be recomputed from totals.

8.5 Synthetic data generation (`greentwin/twin/generate.py`)

1. **Homes:** real EPC records from the three regions (≈ 50–100k per region), raked to the EPC live-table band and type mixes per LA.
2. **Behaviour:** sample setpoint, heating hours, occupants, baseload and post-retrofit rebound from priors tuned so the generated annual totals match **NEED** distributions per archetype (type × age × floor area × region).
3. **Monthly usage:** run the twin (calibrated mode) on each home with its **real monthly weather**, then add meter noise (StudentT, ν = 5). Inject missing months (≈ 8%), estimated reads (≈ 10%) and conflicting records (≈ 1%, e.g. rating A with 400 kWh/m²).
4. **Labels** (`ReferenceLabel`, every rule documented with `rule_id`): - per-retrofit and package per-fuel savings (twin, true parameters) - improvement potential and ranking - feasibility (rules) - rating change (standard mode) - synthetic value with a planted premium, **for method checks only**
5. **Validation report** (`eval/reports/synthetic_validation.md`): - KS / quantiles vs NEED annual consumption per archetype - band / type / kWh/m² vs the EPC live tables per LA - the stated caveat: agreement with synthetic labels is not real-world accuracy

8.6 Load profiles and area labels

- **London smart meters** → `LoadProfileArchetype`: drop households with < 300 days of data; normalise each day to sum 1; k-means (k ≈ 8) per season × day type; keep archetypes with ≥ 50 households. Used by the electricity layer (Part 1). The data are electricity-only, London 2011–14; this is stated as a limitation.
- **DESNZ sub-national LSOA consumption** → `LSOAConsumption`. Suppressed cells (few meters) stay `None`, never 0. These are the labels for the GraphSAGE area model:
- **Graph:** LSOA adjacency from the 2021 boundaries.
- **Features:** EPC stock mix, IMD decile, degree days, off-gas share, mean floor area.
- **Validation:** spatial-block CV by local authority, against an EBM baseline without the graph.

8.7 Commands

```
uv run python scripts/download_epc.py --regions NW SW London      # needs EPC service account (One Login)
uv run python scripts/download_public.py --all                    # NEED, DESNZ LSOA, UCL, PPD, HPI, ONSPD, IMD,
boundaries, live tables
uv run python scripts/download_weather.py --districts data/interim/districts.csv   # Open-Meteo, cached
uv run python -m greentwin.data.build                             # map → split → DuckDB + meta.json
uv run python -m greentwin.data.live_tables                       # EPC live tables → area_epc_stats
uv run python -m greentwin.twin.generate --n 50000 --seed 42      # synthetic usage + labels
uv run pytest -q                                                  # contracts, splits, leakage, reconciliation
```

9. Configuration files (`contracts/`)

File	Owner part	Contents
`score_mapping.json`	3	Fixed weights, monotone sub-score mappings, intensity definition, overlap justification
`feasibility_rules.json`	5	Rule vetoes / downgrades with reason codes; ordering groups
`retrofit_catalogue.json`	2	Retrofits, `twin_edits`, dependencies, sourced costs + `cost_status`, grants
`sap_estimator.json`	2	Post-retrofit rating method (SAP 2012 ECF, anchoring, band thresholds)
`emission_factors.json`	2	DESNZ 2025 factors (electricity 0.19553, gas 0.18296 gross CV) with row ids
`tariffs.json`	2	Ofgem Q4 2026 rates; time-of-use entry unsourced until cited
`green_profits_assumptions.json`	Orchestrator	Scenario rule for take-up; every coefficient a labelled assumption

10. Testing, evaluation, fallbacks

Part	Must-pass tests	Held-out metrics (vs baseline)
0	contracts validate; no id spans splits; no leakage fields in features; mapper coverage ≥ 95%; our EPC counts per LA × quarter reconcile with live table D1	—
1	twin monotone sweep; SH1–SH3 within ±15% of metered; `estimated=True` on element heat loss	calibration coverage ≈ 90%, CRPS, $\hat{R}$ (vs degree-day OLS, LightGBM quantile); standard-mode kWh/m² vs EPC D3 by LA; masked-month MAE; capture room-area error + tag accuracy
2	per-fuel signs; package = one run; potential null at baseline 0; rating monotone	EBM MAE/MAPE + coverage (vs NEED median, XGBoost); rating Δ on EPC before/after pairs; ranking NDCG; precision@k vs EPC recommendations; Pareto hypervolume
3	weights sum to 1; monotone sub-scores; every heating type mapped	— (deterministic); band-probability calibration
4	wording always present; uses current value, never a sale price	coverage + pinball (vs OLS without rating); β vs published
5	rule veto beats Jev; validator rejects unsupported numbers; works with no keys	routing accuracy ≥ 95% above τ; feasibility precision/recall; groundedness ≥ 90%, accuracy ≥ 90%, unsupported < 5% on ≥ 30 scenarios

Fallbacks:

Missing	Replacement
No LLM key	Template explainer
No Jev key	Claude structured output → rules
No EPC API	Cached sample files
Calibration failure	`prior_only` + warning
Unsourced cost / factor / tariff	`null` + reason
No WebGL	2D view